XML Signature

Web Security

Juraj Somorovsky

Summer Semester 2024

Paderborn University

This lecture

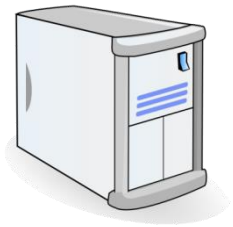
1. Web technologies
2. Web attacks
3. **XML and Single Sign-On (SAML)**
4. JSON and OIDC / OAuth
5. JSON Web Services

Denial of Service
Billion Laugh Attack (Klein, 2002)

```
<!DOCTYPE data [
        <!ENTITY a "dos" >
        <!ENTITY b "&a;&a;&a;">
        <!ENTITY c "&b;&b;&b;">
]>
<data>dosdosdosdosdosdosdosdosdos</data>
```

XML External Entity Attack (XXE)

```
<!DOCTYPE svg [
<!ENTITY file SYSTEM "file:///etc/passwd">
]>
<svg xmlns="http://www.w3.org/2000/svg">
        <rect width="500" height="500"
                    style="fill:rgb(255,0,0);"/>
        <text x="10" y="30">&file;</text>
</svg>
```



Server

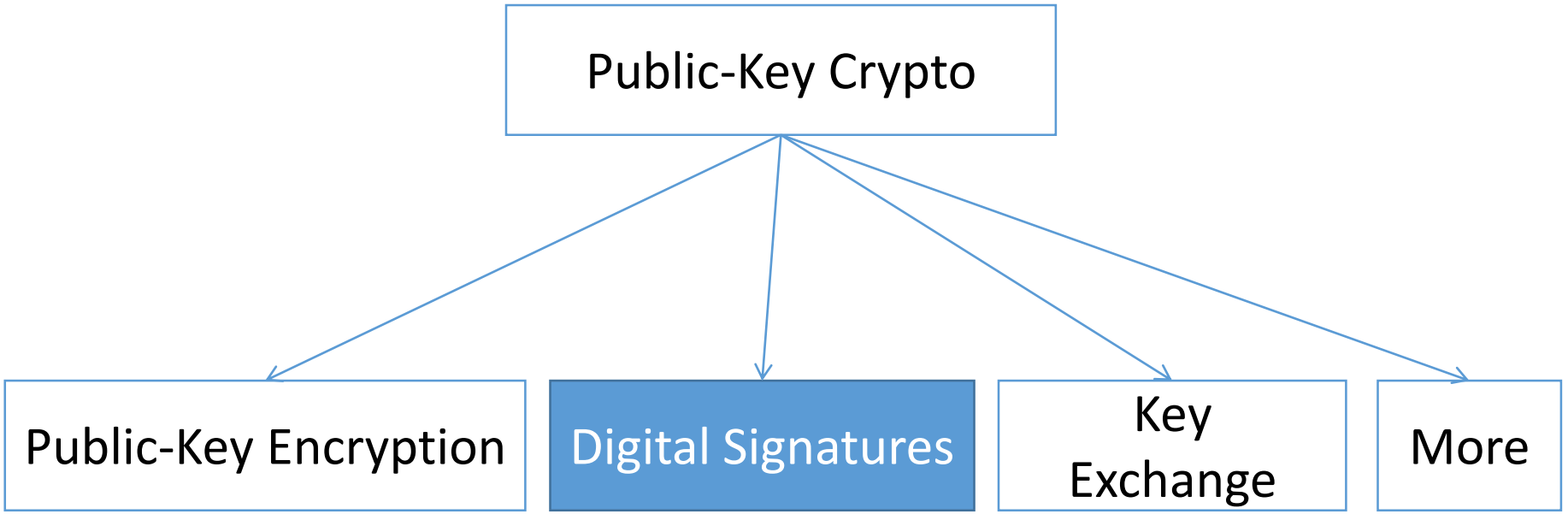
Image/PNG:

```
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/bin/false
...
```

Overview

1. Cryptography Basics
2. XML Signature
3. HMAC Truncation Attack
4. XML Signature Wrapping
5. XML Signature Exclusion
6. SAML-based Single Sign-On

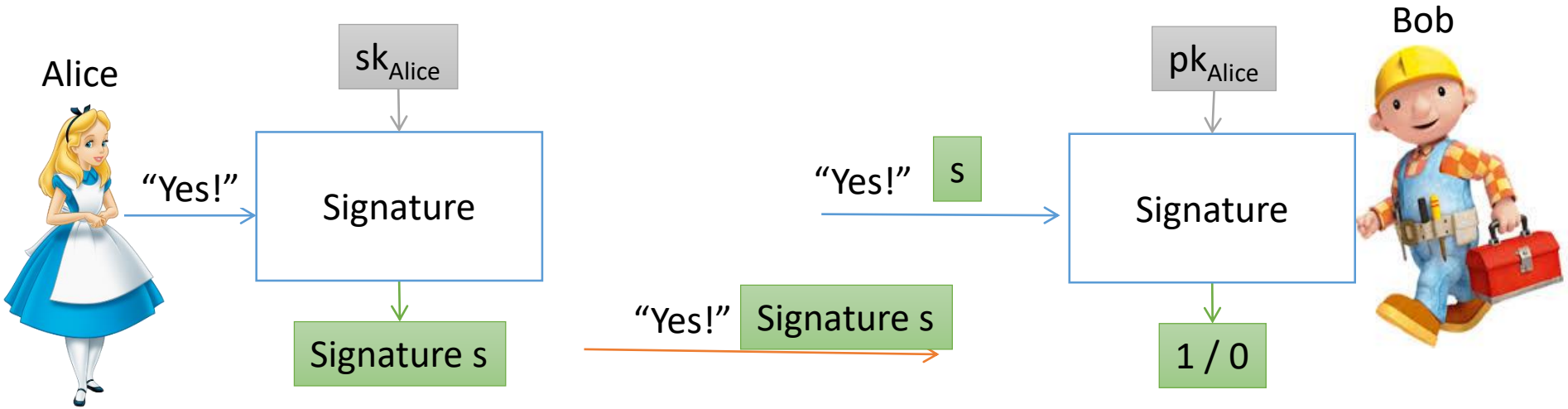
Overview of Public Key Cryptography



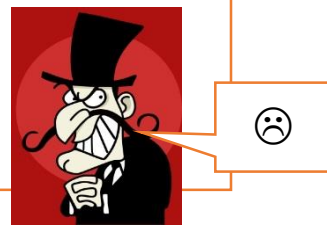
Two keys:

- Secret/private key is kept private: sk_{Alice}
- Public key for anybody: pk_{Alice}

Digital Signatures



- **Two keys:**
  - Alice has public and private key (pk_{Alice}, sk_{Alice})
  - Bob has public key of Alice (pk_{Alice})
- Examples: RSA, ECDSA signatures



Beyond Digital Signatures

Can we protect integrity and authenticity with symmetric key algorithms?

Yes - HMACs

XML Security

- W3C Standard for securing XML messages
- XML Encryption: protects confidentiality
- XML Signature: protects authenticity and integrity
- Very flexible

```
<PaymentInfo>
  <Name>John Smith</Name>
  <CreditCard Limit='5,000'>
    <Number>4019 ...5567</Number>
    <Issuer>Example Bank</Issuer>
    <Expiration>04/02</Expiration>
  </CreditCard>
</PaymentInfo>
```

XML Security Areas

- Financial services:
  - Electronic Banking Internet Communication (EBICS)
- Healthcare:
  - Australian eHealth Technical Specification
- Governmental services:
  - ID cards in Estonia, Germany, Hungary, …
- System integration, firewalls

Dresdner Bank
Die Beraterbank

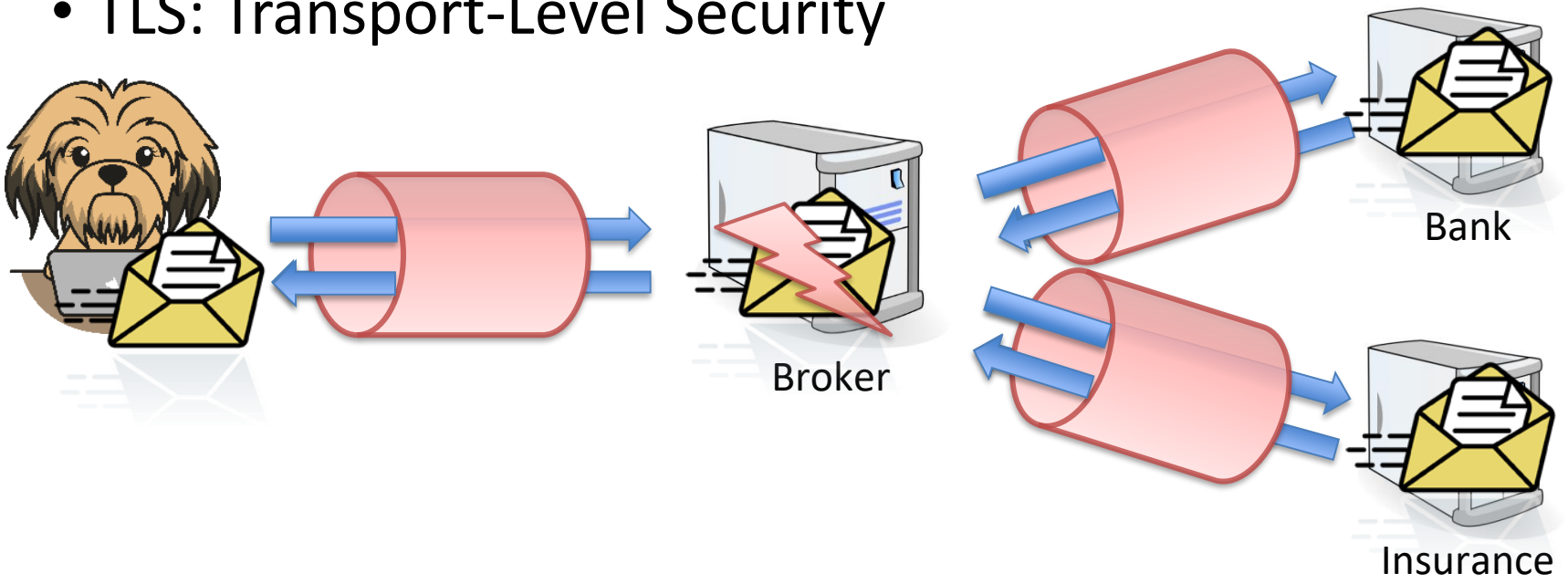
COMMERZBANK

Sparkasse

IBM WebSphere

Motivation – XML Security

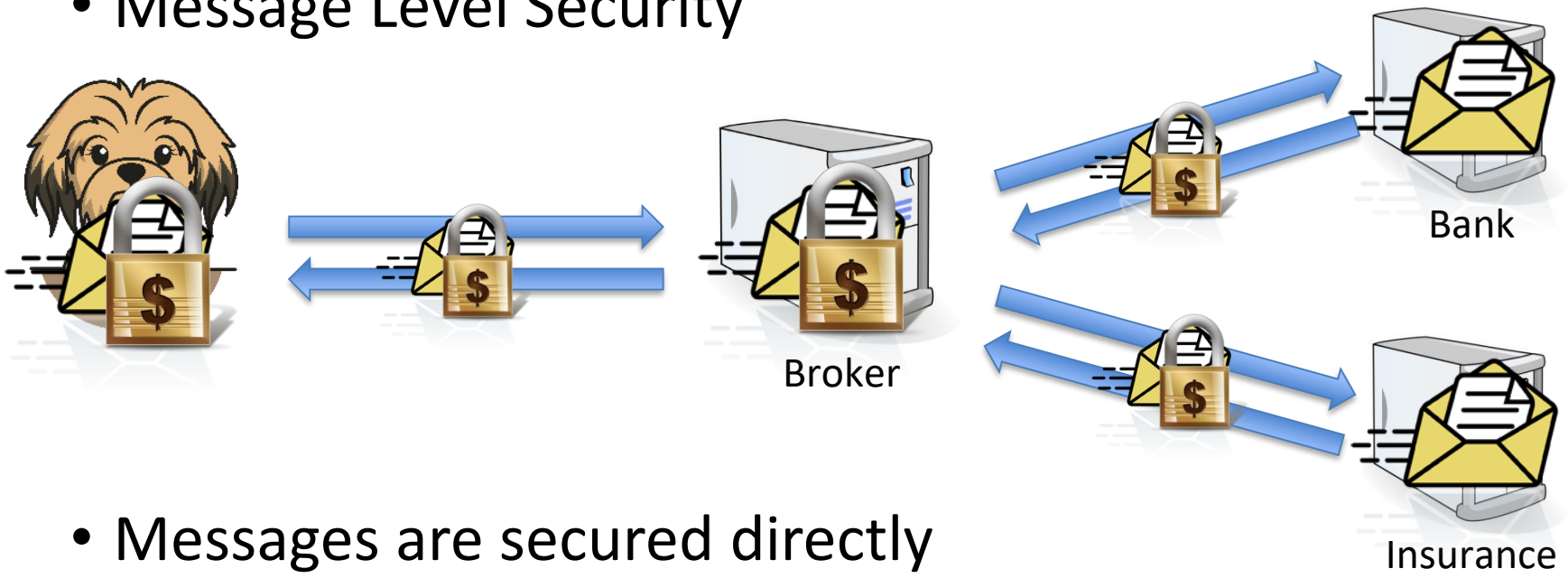
- TLS: Transport-Level Security



- Messages are only secured during transport

Motivation – XML Security

- Message Level Security



- Messages are secured directly
- TLS not needed
- Use of XML Signature, XML Encryption

Overview

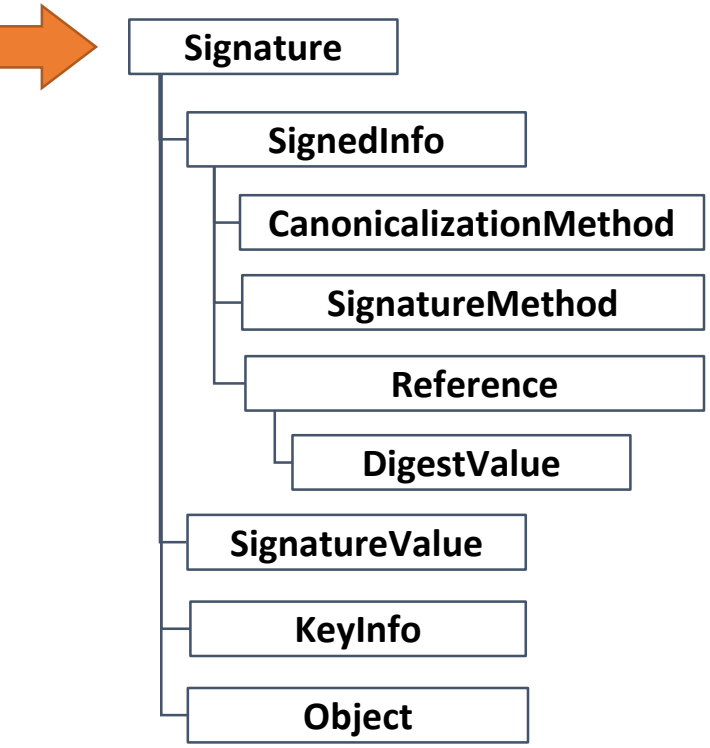
1. Cryptography Basics
2. XML Signature
3. HMAC Truncation Attack
4. XML Signature Wrapping
5. XML Signature Exclusion
6. SAML-based Single Sign-On

Very boring content!

XML Signature

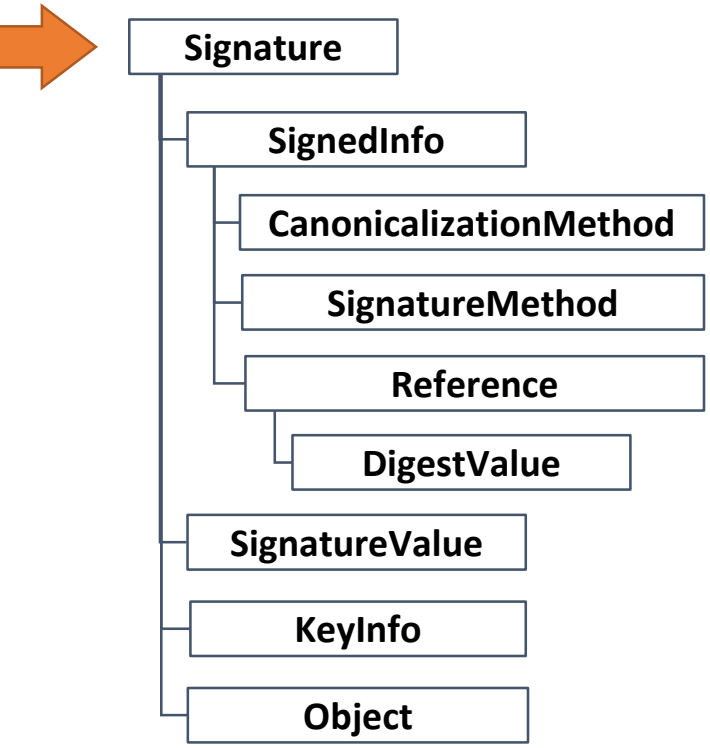
```
<Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
  <SignedInfo>
    <CanonicalizationMethod Algorithm="…/REC-xml-c14n"/>
    <SignatureMethod Algorithm="…#rsa-sha256"/>
    <Reference URI="#2">
      <Transforms>
       <Transform Algorithm="…#enveloped-signature"/>
      </Transforms>
      <DigestMethod Algorithm="…xmldsig#sha256"/>
      <DigestValue>rXs1F7C/…=</DigestValue>
    </Reference>
  </SignedInfo>
  <SignatureValue>…</SignatureValue>
  <KeyInfo>
    <X509Data>
    …
    </X509Data>
  </KeyInfo>
</Signature>
```

XML Signature in Detail



- Latest version: XML Signature 2.0
- Link: https://www.w3.org/TR/xmldsig-core2/

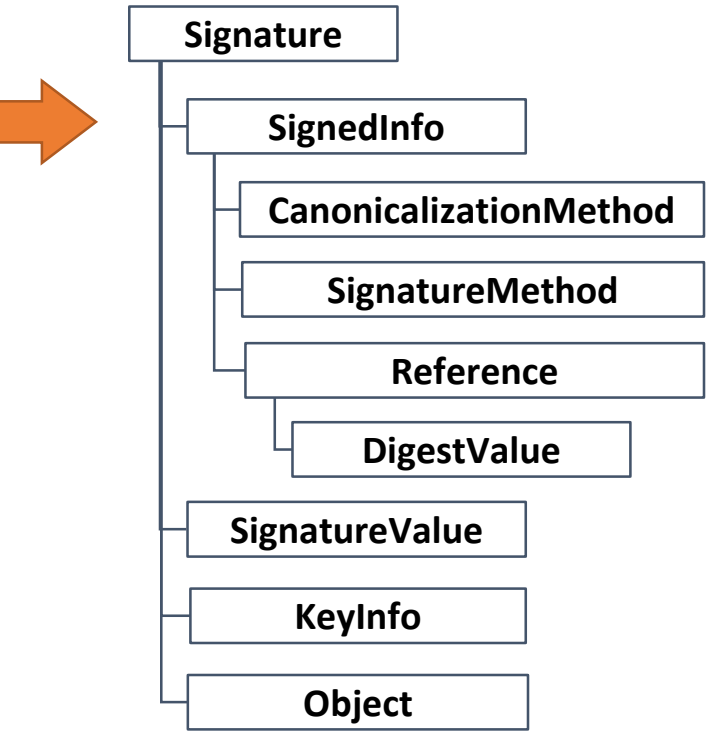
XML Signature in Detail



<Signature/>

- Child element sequence
  - SignedInfo
  - SignatureValue
  - Optional: KeyInfo
  - Optional: Object

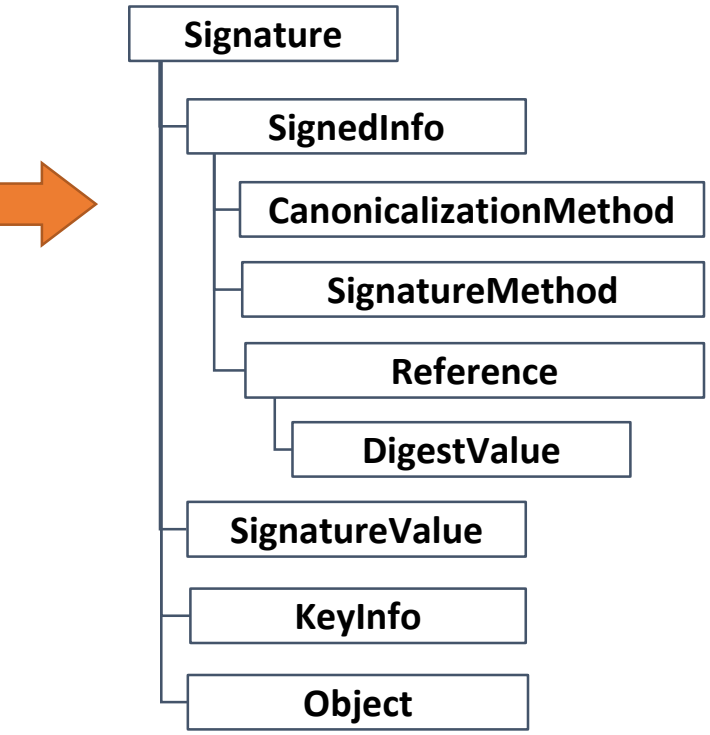
XML Signature in Detail



<SignedInfo/>

- Child element sequence
  - CanonicalizationMethod
  - SignatureMethod
  - One ore more Reference elements

XML Signature in Detail



<CanonicalizationMethod/>

- Mandatory Attribute **Algorithm**
  - Inclusive c14n
    - With comments
    - Without comments
  - Exclusive c14n
    - With comments
    - Without comments

XML Canonicalization (c14n)

- Idea: XML Signature signs „sematic content“
  - E.g. changing attribute order should not invalidate the signature

Other ideas when document modification does not change the semantic content?

```
<?xml version="1.0"?>  +cr++lf+
<List id =  '123'          >  +cr++lf+
    <Element1>  Hallo  </Element1>+cr++lf+
    <Element2/>+cr++lf+
</List>+cr++lf+
```

XML Canonicalization (c14n)

- Idea: XML Signature signs „sematic content“
  - E.g. changing attribute order should not invalidate the signature
- c14n does basically:
  - Order Attributes, set default values
  - Remove unnecessary whitespaces in element declarations (NOT in element content!)
  - Convert single quotes to double quotes
  - Convert <x/> to <x></x>

Canonicalization in Detail

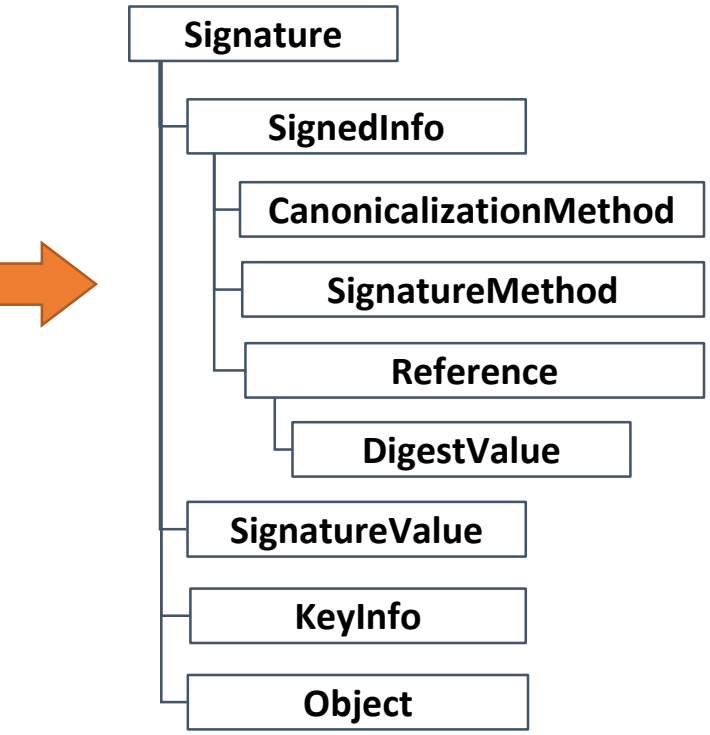
- http://www.w3.org/TR/xml-c14n#Terminology
  - The document is encoded in UTF-8
  - Line breaks normalized to #xA on input, before parsing
  - Attribute values are normalized, as if by a validating processor
  - Character and parsed entity references are replaced
  - CDATA sections are replaced with their character content
  - The XML declaration and document type declaration (DTD) are removed
  - Empty elements are converted to start-end tag pairs
  - Whitespace outside of the document element and within start and end tags is normalized
  - All whitespace in character content is retained (excluding characters removed during line feed normalization)
  - Attribute value delimiters are set to quotation marks (double quotes)
  - Special characters in attribute values and character content are replaced by character references
  - Superfluous namespace declarations are removed from each element
  - Default attributes are added to each element
  - Lexicographic order is imposed on the namespace declarations and attributes of each element

Canonicalization Example

```
<?xml version="1.0"?>  +cr++lf+
<List id =  '123'          >  +cr++lf+
    <Element1>  Hallo  </Element1>+cr++lf+
    <Element2/>+cr++lf+
</List>+cr++lf+
```

```
<List id="123">  +lf+
    <Element1>  Hallo  </Element1>+lf+
    <Element2></Element2>+lf+
</List>+lf+
```

XML Signature in Detail

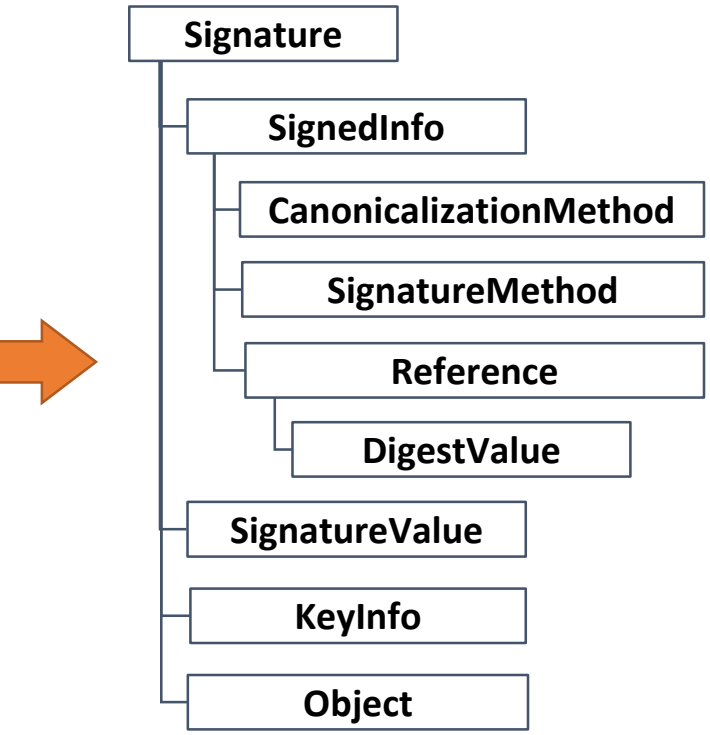


<SignatureMethod/>

- Specifies Signature/MAC Method
- Algorithms:
  - DSAwithSHA256
  - HMAC-SHA256 (Message Authentication Code!)
  - RSAwithSHA256

Use of SHA-256 is strongly recommended over SHA-1 because recent advances in cryptanalysis …. Therefore, SHA-1 support is REQUIRED in this specification only for backwards-compatibility reasons. (XML Signature 2 spec)

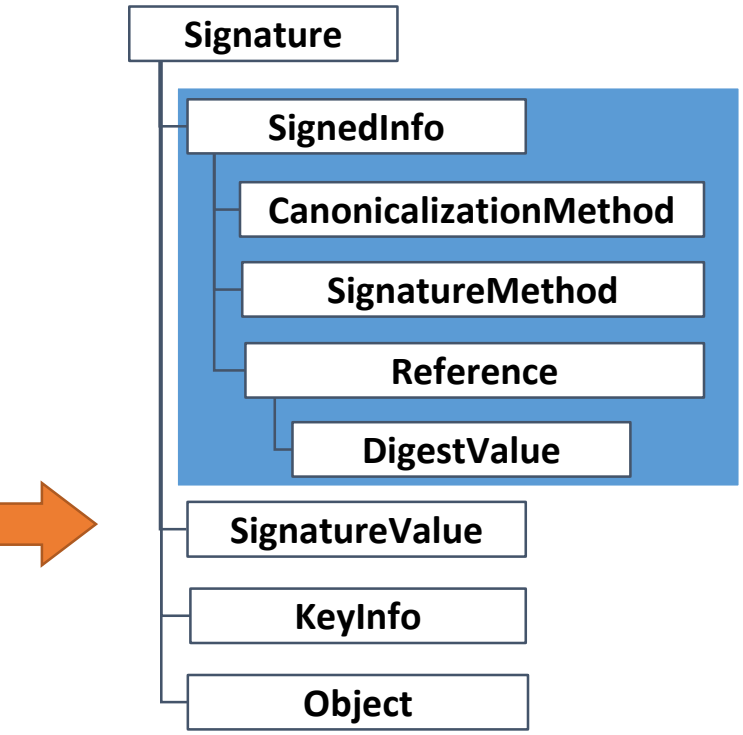
XML Signature in Detail



1..n <Reference/> specify the signed content

- **URI attribute**
  - **#id-of-signed-content**
  - **“” ←whole document**
- Transformations possible
  - Canonicalization
  - XPath Filter
  - Many more
    - e.g. Base64, XSLT

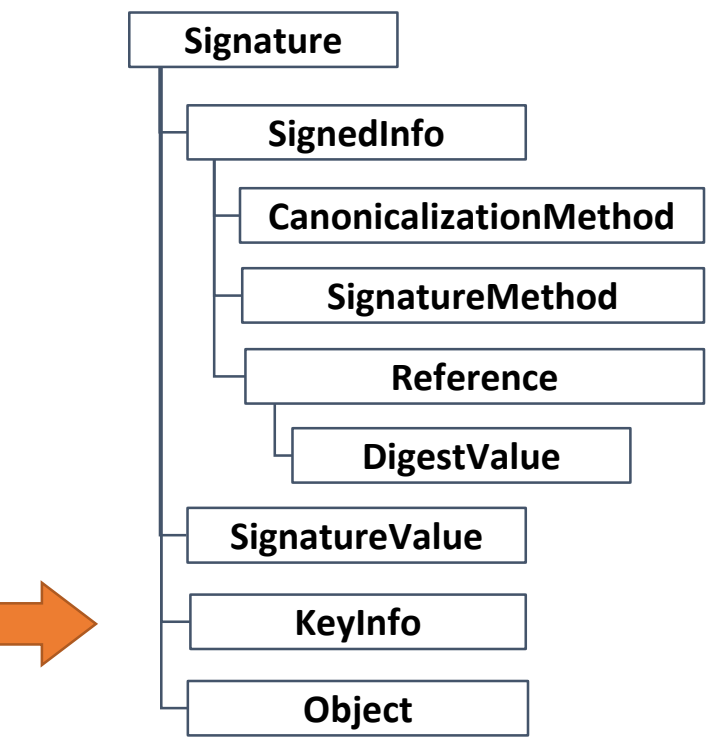
XML Signature in Detail



<SignatureValue/>

- Contains Base64 coded signature value
- Computed over the SignedInfo element

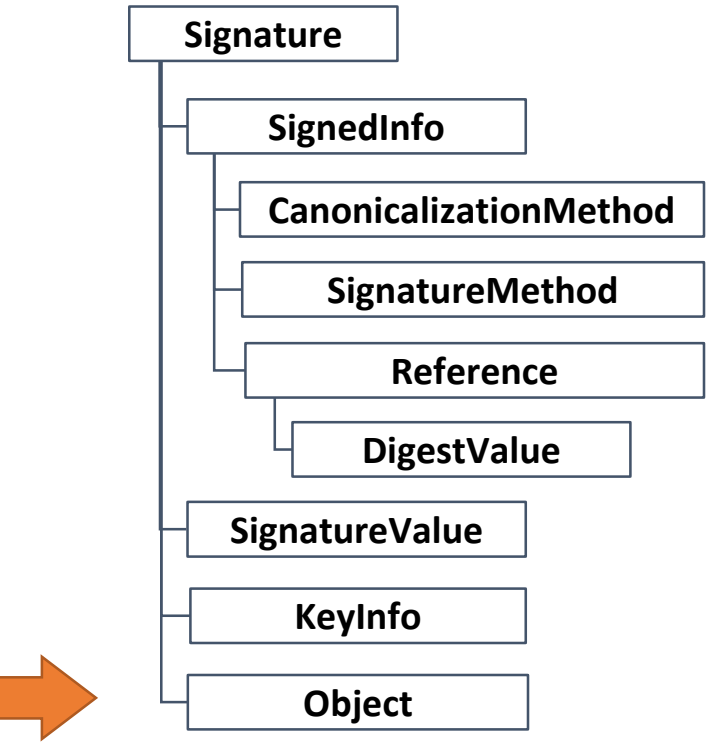
XML Signature in Detail



<KeyInfo/>

- Optional
- Contains key material
  - X509 certificate
  - PGP Data
  - …
- Attention:
  Validate key origin

XML Signature in Detail

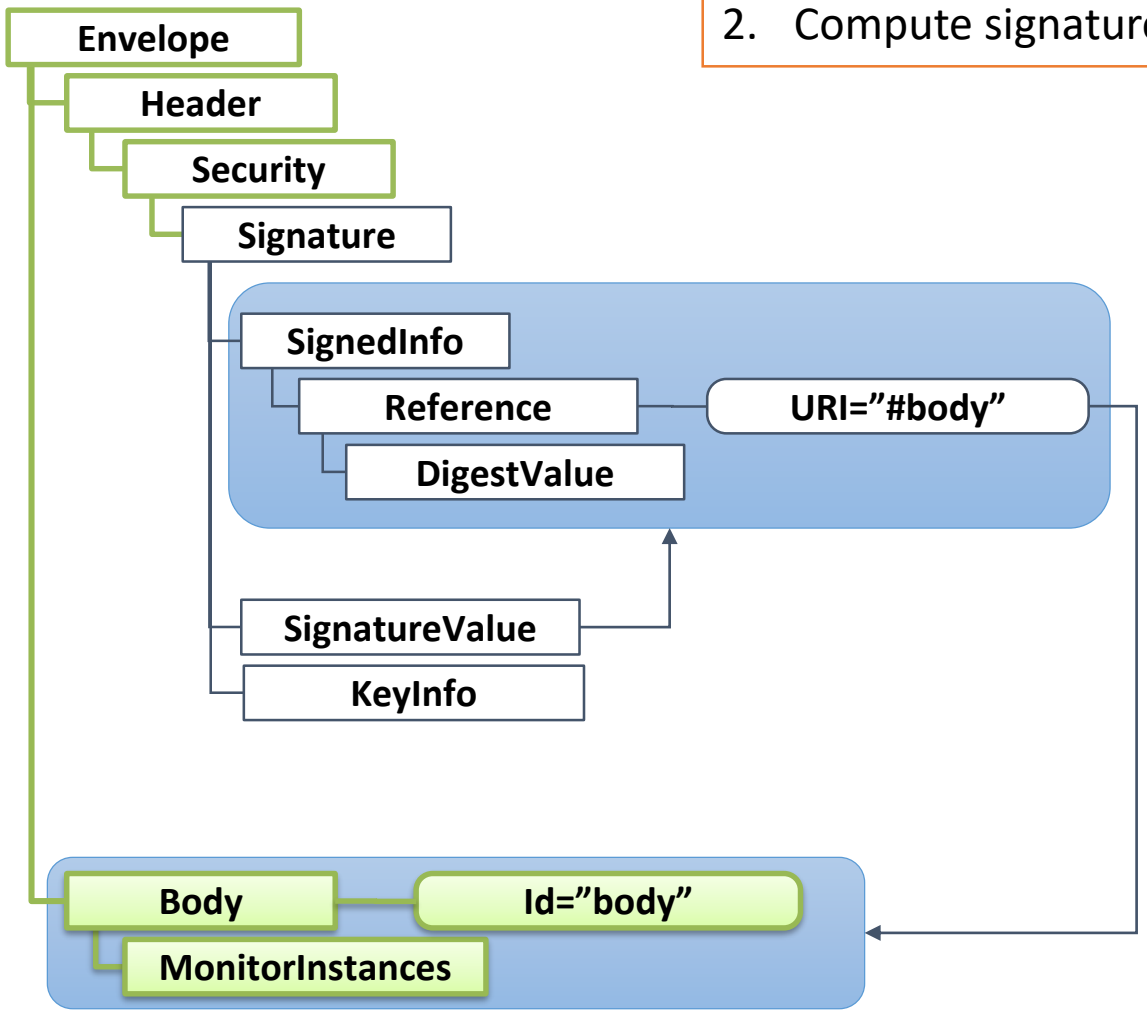


<Object/>

- Optional
- Can contain arbitrary data
- Commonly used for enveloping signatures (see later)

XML Signature

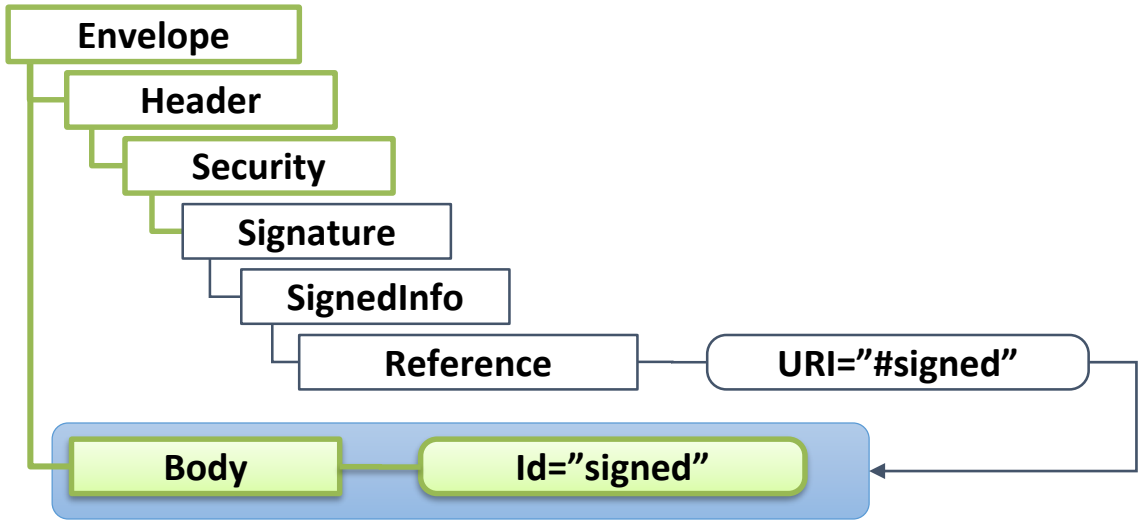
Summary:
1. Compute digest/hash over the referenced element
2. Compute signature value over SignedInfo



DEMO

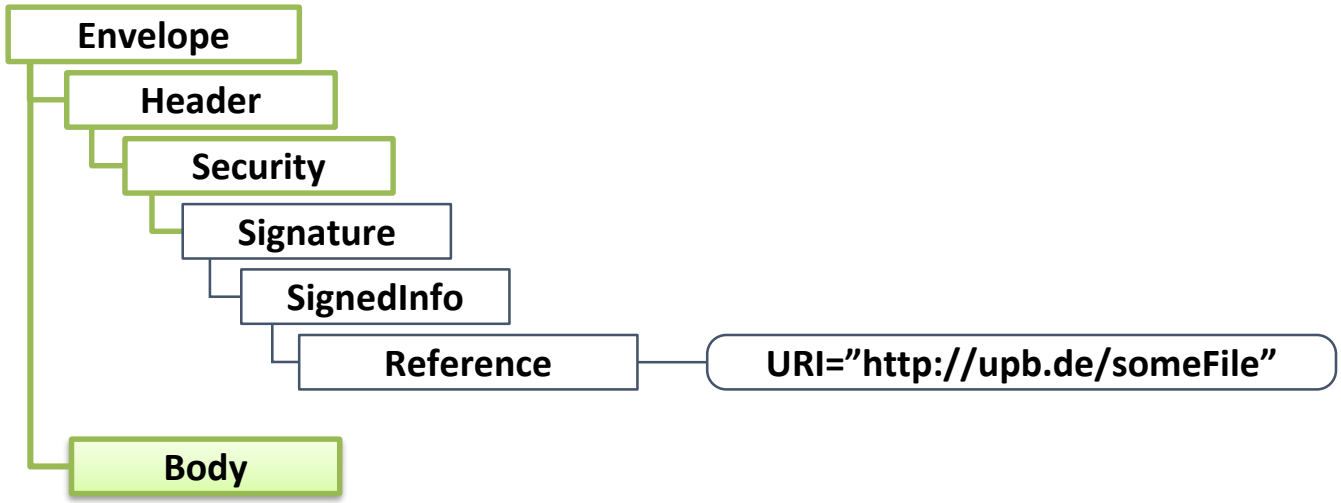
There are 3 types of XML Signatures

Detached XML Signature



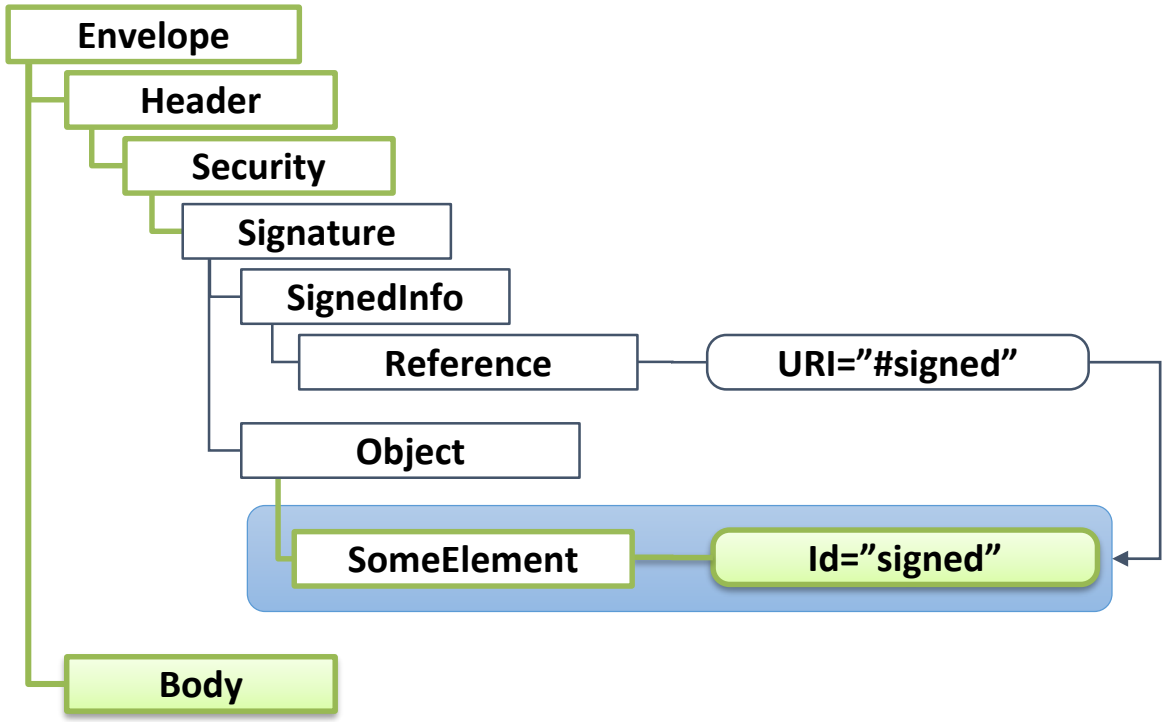
- Signed Element is detached (no child, no ancestor node to <Signature/>)

Detached XML Signature (2)



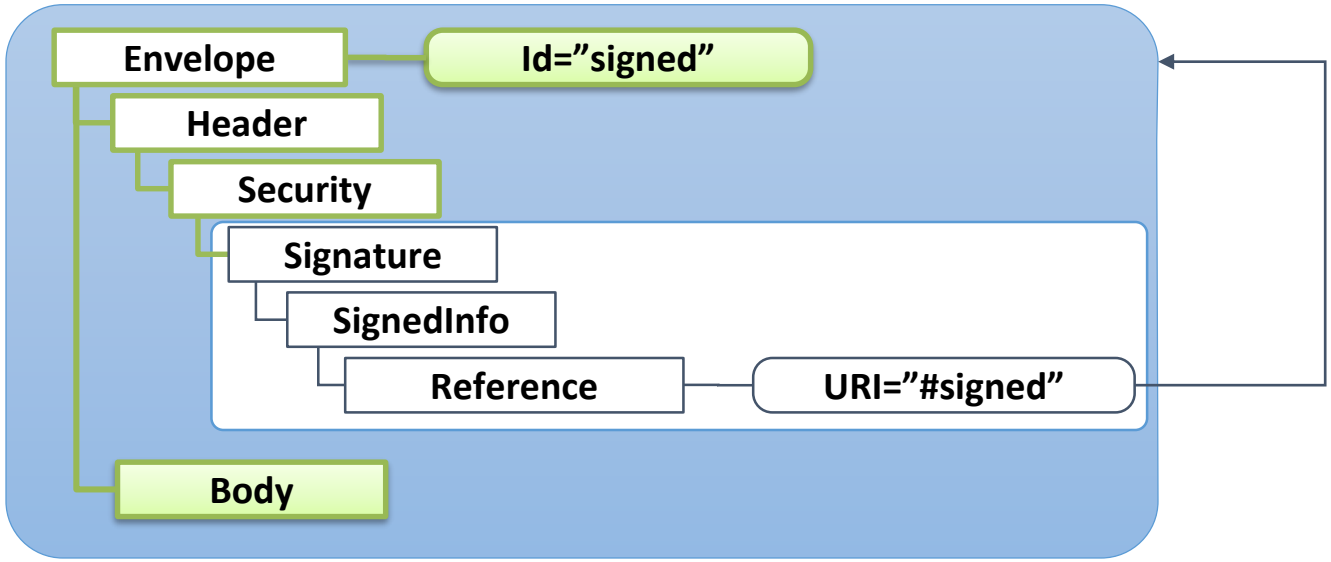
- Signed Element is external

Enveloping XML Signature



- Signed Element is a child of <Signature/>

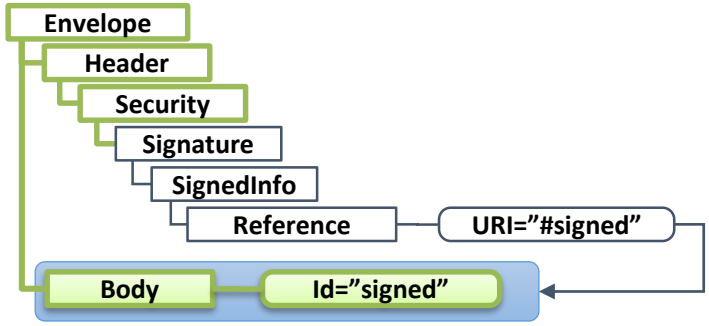
Enveloped XML Signature



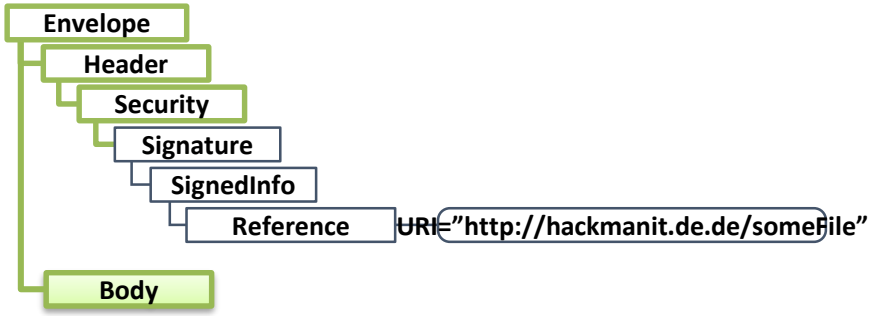
- Signed Element is an ancestor of <Signature/>
- Signature element is excluded from digest calculation (Enveloped Signature Transformation)

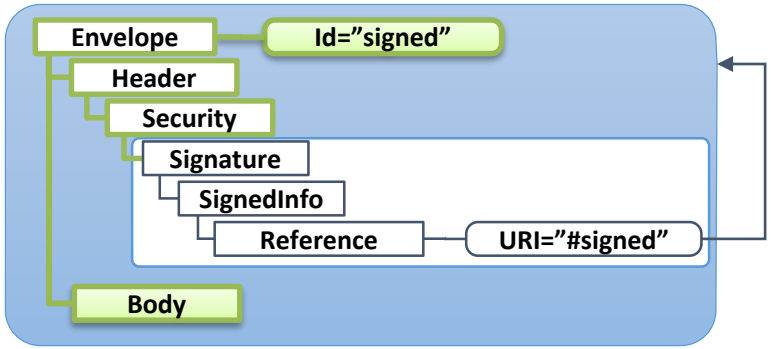
Signature Types - Overview

Detached 1

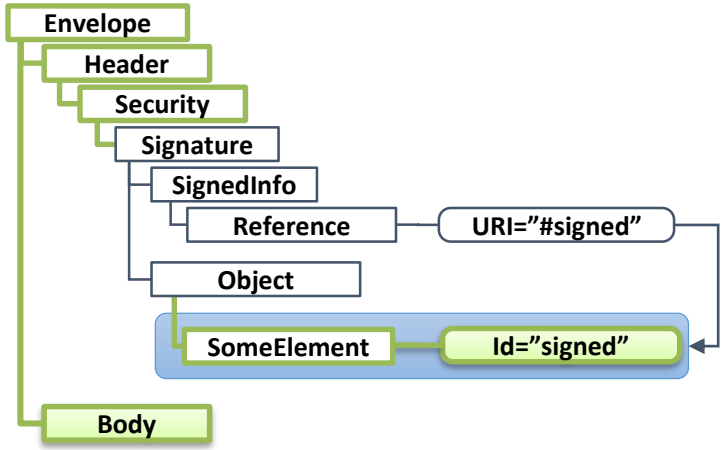


Detached 2





Enveloped



Enveloping

Questions

Questions

- Is the document still valid if you …
  - Modify the **timestamp**
  - Modify the **user**
  - Pretty print the document

A: Yes, B: No

https://pingo.coactum.de/633763

Overview

1. Cryptography Basics
2. XML Signature
3. HMAC Truncation Attack
4. XML Signature Wrapping
5. XML Signature Exclusion
6. SAML-based Single Sign-On

Two types of XML Signatures

- Public key XML Signatures:
  - `http://www.w3.org/2000/09/xmldsig#dsa-sha1`
  - `http://www.w3.org/2000/09/xmldsig#rsa-sha1`
- Secret key XML Signatures (HMACs):
  - `http://www.w3.org/2000/09/xmldsig#hmac-sha1`

Read this carefully…

- From the XML Signature spec:

The HMAC algorithm (RFC2104…) takes the truncation length in bits as a parameter; if the parameter is not specified then all the bits of the hash are output. An example of an HMAC SignatureMethod element:

```
<SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#hmac-sha1">
    <HMACOutputLength>128</HMACOutputLength>
</SignatureMethod>
```

The output of the HMAC algorithm is ultimately the output (possibly truncated) of the chosen digest algorithm. This value shall be base64 encoded in the same straightforward fashion as the output of the digest algorithms.

Can you spot a problem?

Read this carefully…

- From the XML Signature spec:

The HMAC algorithm (RFC2104…) takes the truncation length in bits as a parameter; if the parameter is not specified then all the bits of the hash are output. An example of an HMAC SignatureMethod element:

```
<SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#hmac-sha1">
    <HMACOutputLength>128</HMACOutputLength>
</SignatureMethod>
```

The output of the HMAC algorithm is ultimately the output (possibly truncated) of the chosen digest algorithm. This value shall be base64 encoded in the same straightforward fashion as the output of the digest algorithms.

HMAC Truncation

- How to:
  1. Define an arbitrary document
  2. Create Signature and Reference elements
  3. Create a random SignatureValue
  4. Set HMACOutputLength to **1**

```
<SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#hmac-sha1">
      <HMACOutputLength> 1 </HMACOutputLength>
</SignatureMethod>
```

  5. With a probability of 50 % the message is valid

HMAC Truncation

- Author: Thomas Roessler
- Surprisingly, many systems and frameworks affected
- It took **half a year** to patch all the systems and release a press report
- Countermeasure: Check appropriate HMAC length

http://www.w3.org/blog/2009/07/hmac-truncation-in-xml-signatu

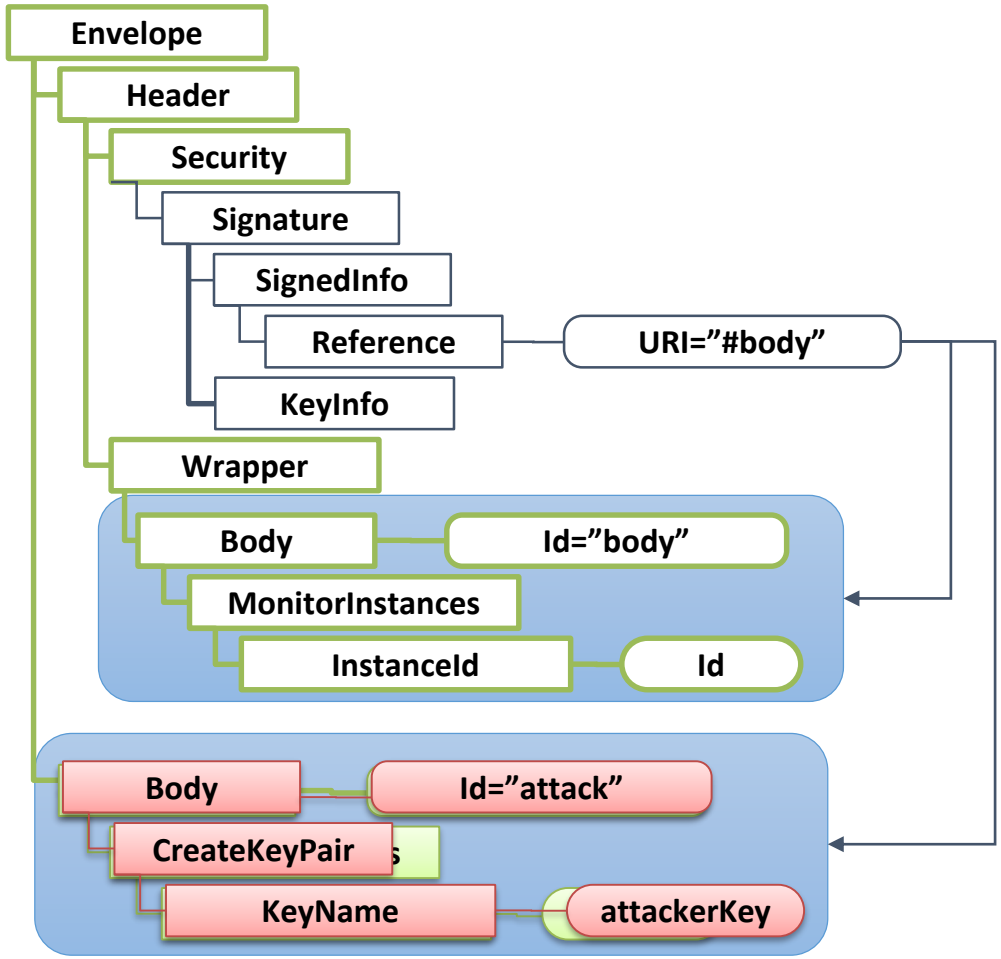
Questions

https://www.comedywildlifephoto.com/

Overview

1. Cryptography Basics
2. XML Signature
3. HMAC Truncation Attack
4. XML Signature Wrapping
5. XML Signature Exclusion
6. SAML-based Single Sign-On

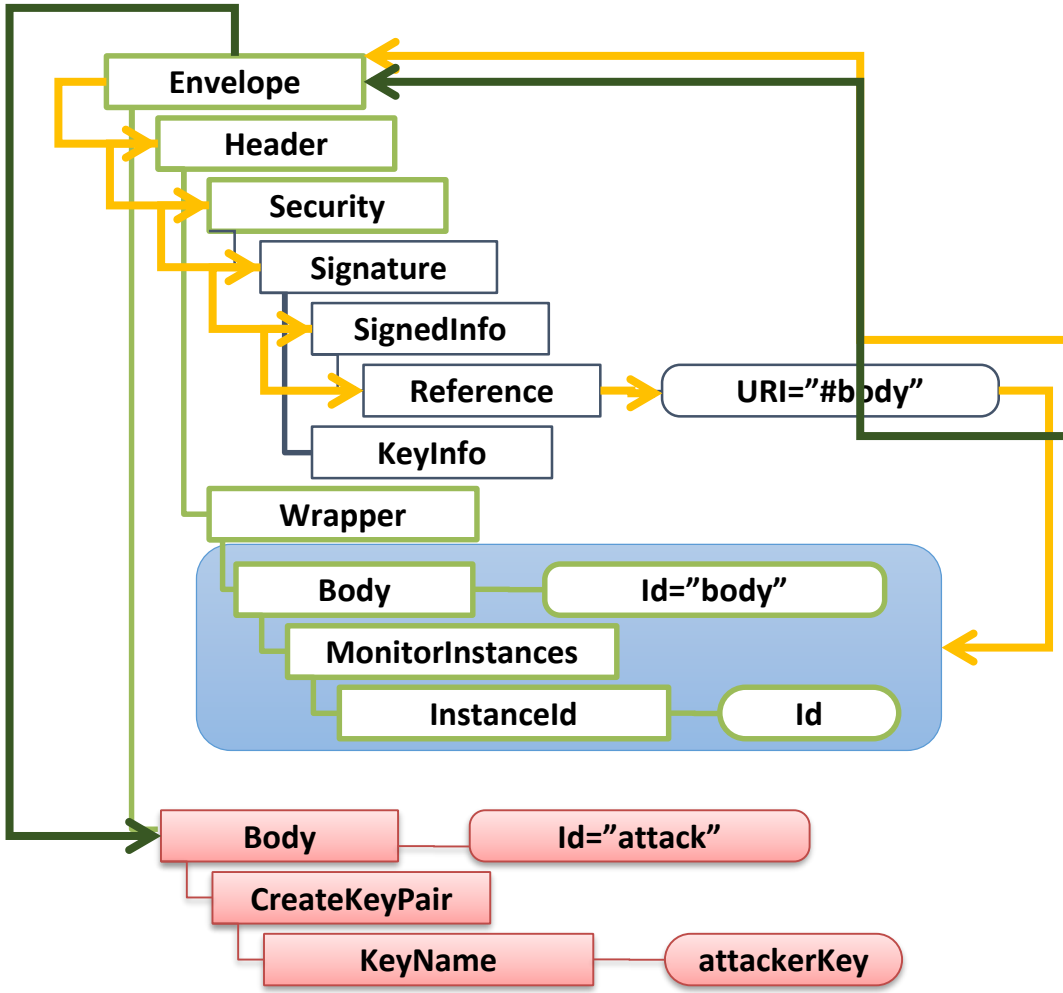
XML Signature Wrapping

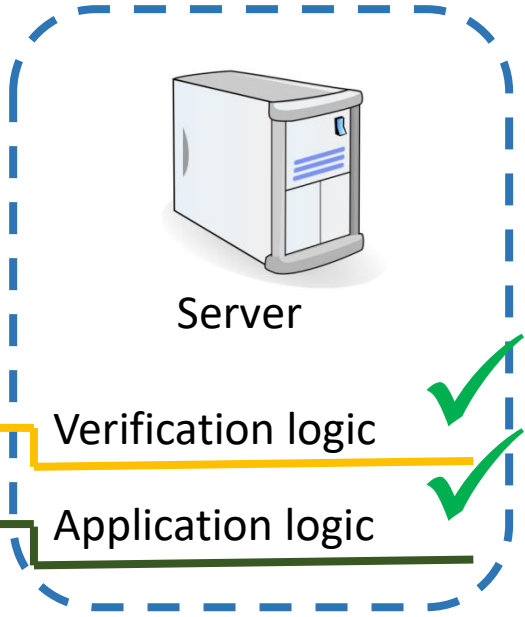


McIntosh and Austel. XML Signature Element Wrapping attacks, 2005

Why does the attack work?

XML Signature Wrapping



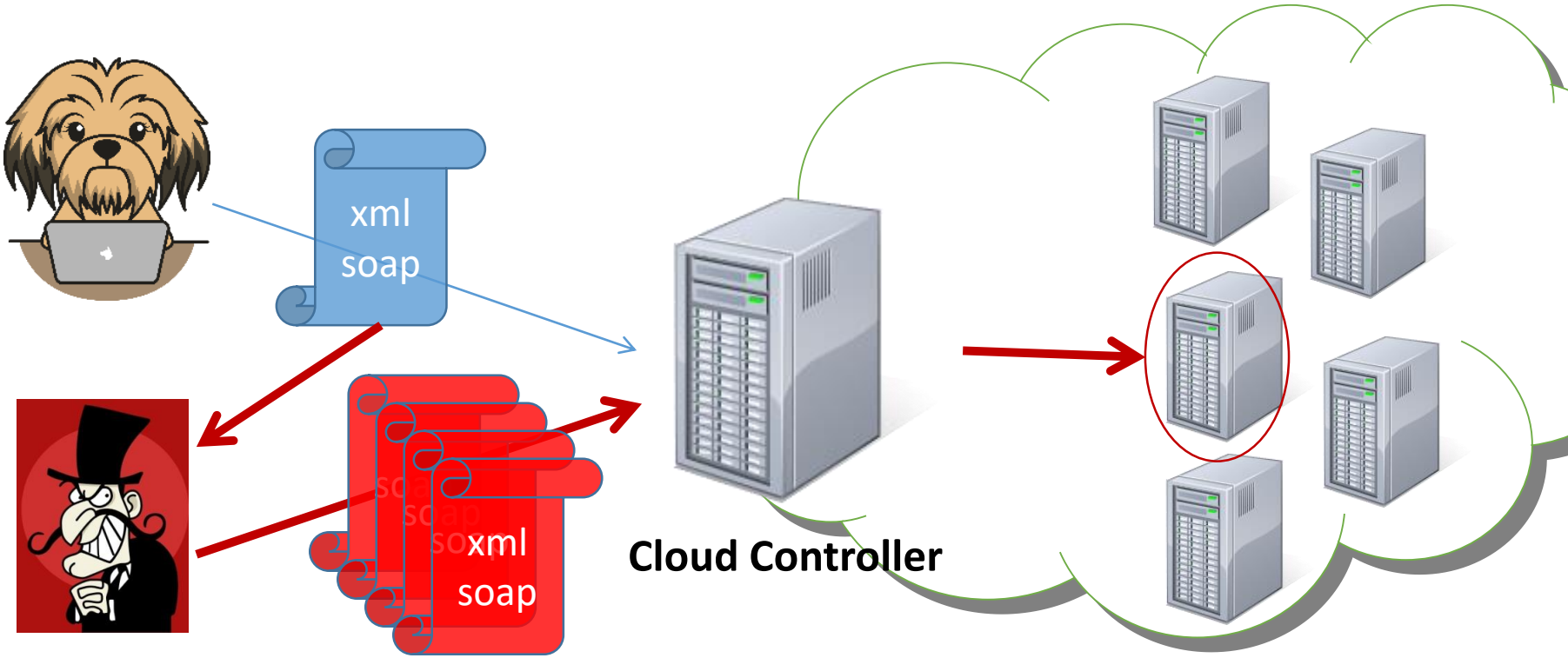


DEMO

Is this a theoretical attack?

XML Signature Wrapping

- Attacks on Amazon EC2 / Eucalyptus clouds
- Juraj Somorovsky, Mario Heiderich, Meiko Jensen, Jörg Schwenk, Nils Gruschka, Luigi Lo Iacono: **All Your Clouds Are Belong to Us – Security Analysis of Cloud Management Interfaces** - CCSW 2011.

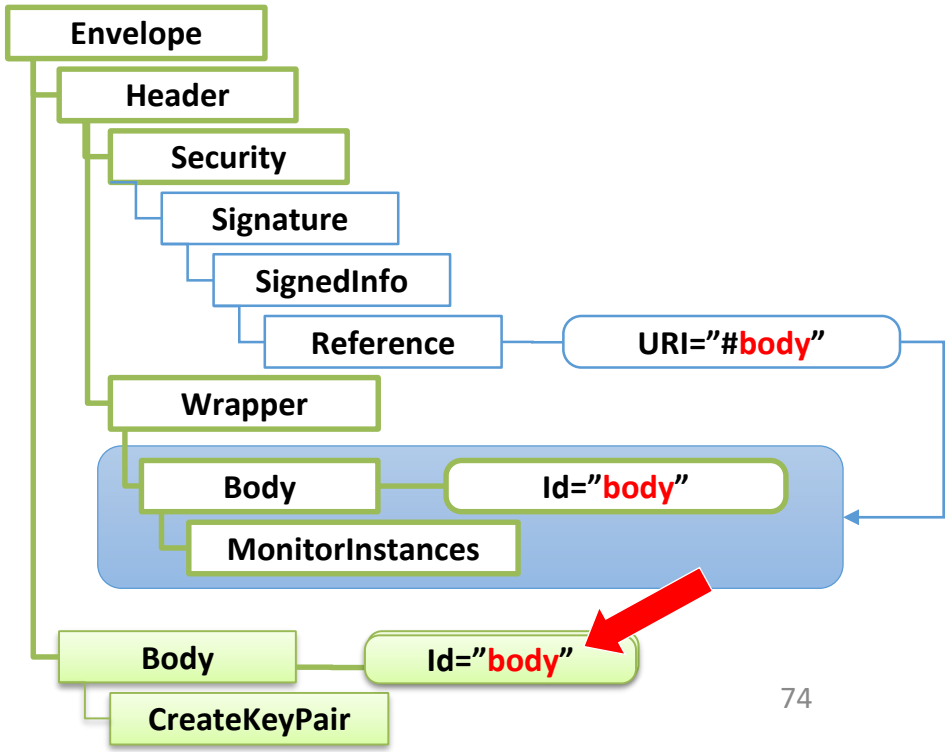


Amazon EC2 Signature Wrapping

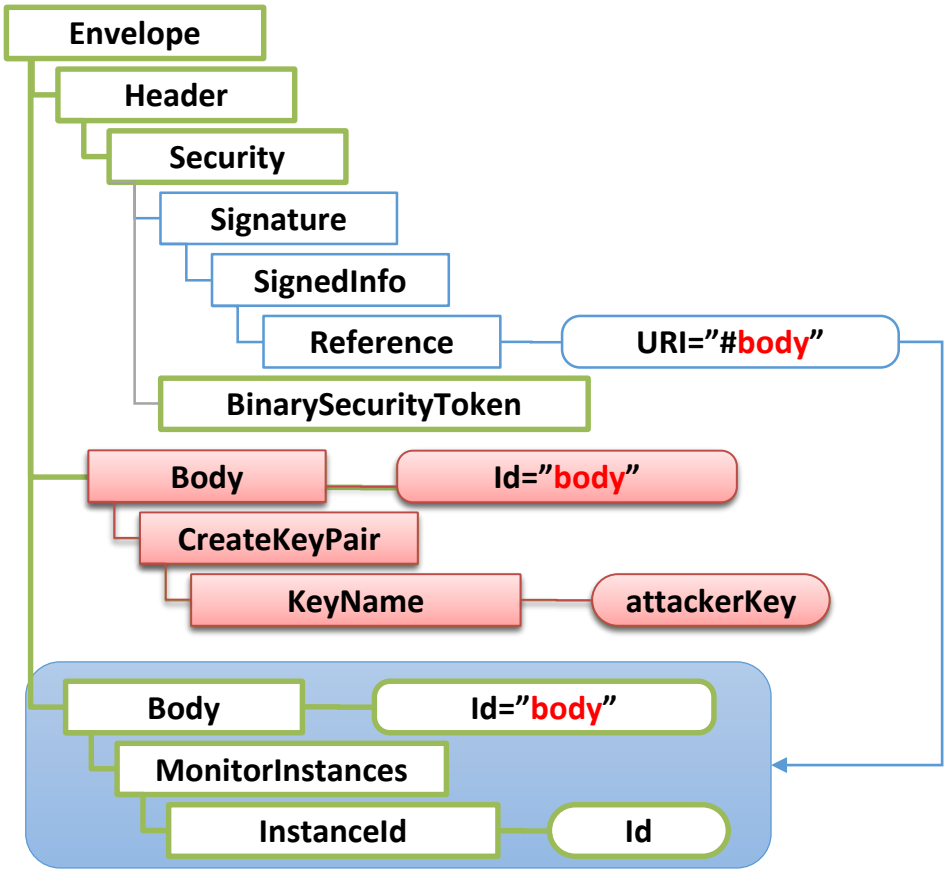
- Amazon checks, whether the Id of the signed element equals to the Id of the processed element
- Attack from McIntosh & Austel does not work
- But what happens if we use two elements with the same Id?

- Which element is used

  for signature validation?
- Which for function execution?



Amazon EC2 Signature Wrapping



Duplicate the Body element:

- Function invocation from the first Body element
- Signature verification over the second Body element

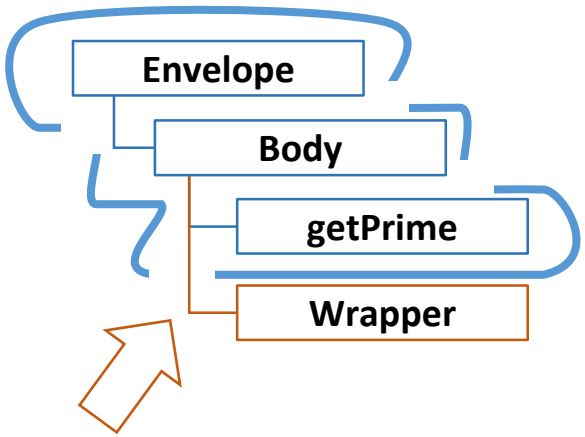
Questions

Does XML Schema validation prevent these attacks?

A: Yes
B: No
C: It depends

https://pingo.coactum.de/633763

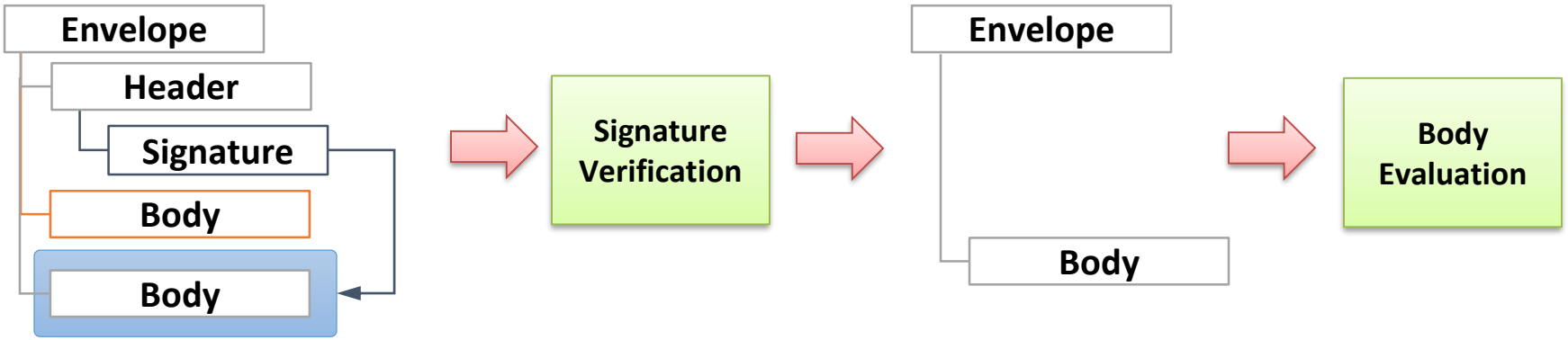
Countermeasure: XML Schema Validation Does NOT Help



- Describe XML tree structure as precise as possible
- Extension points:
  - **On the effectiveness of XML Schema validation for countering XML Signature wrapping attacks.** Meiko Jensen, Christopher Meyer, Juraj Somorovsky, und Jörg Schwenk (2011)

Countermeasure: See-What-Is-Signed

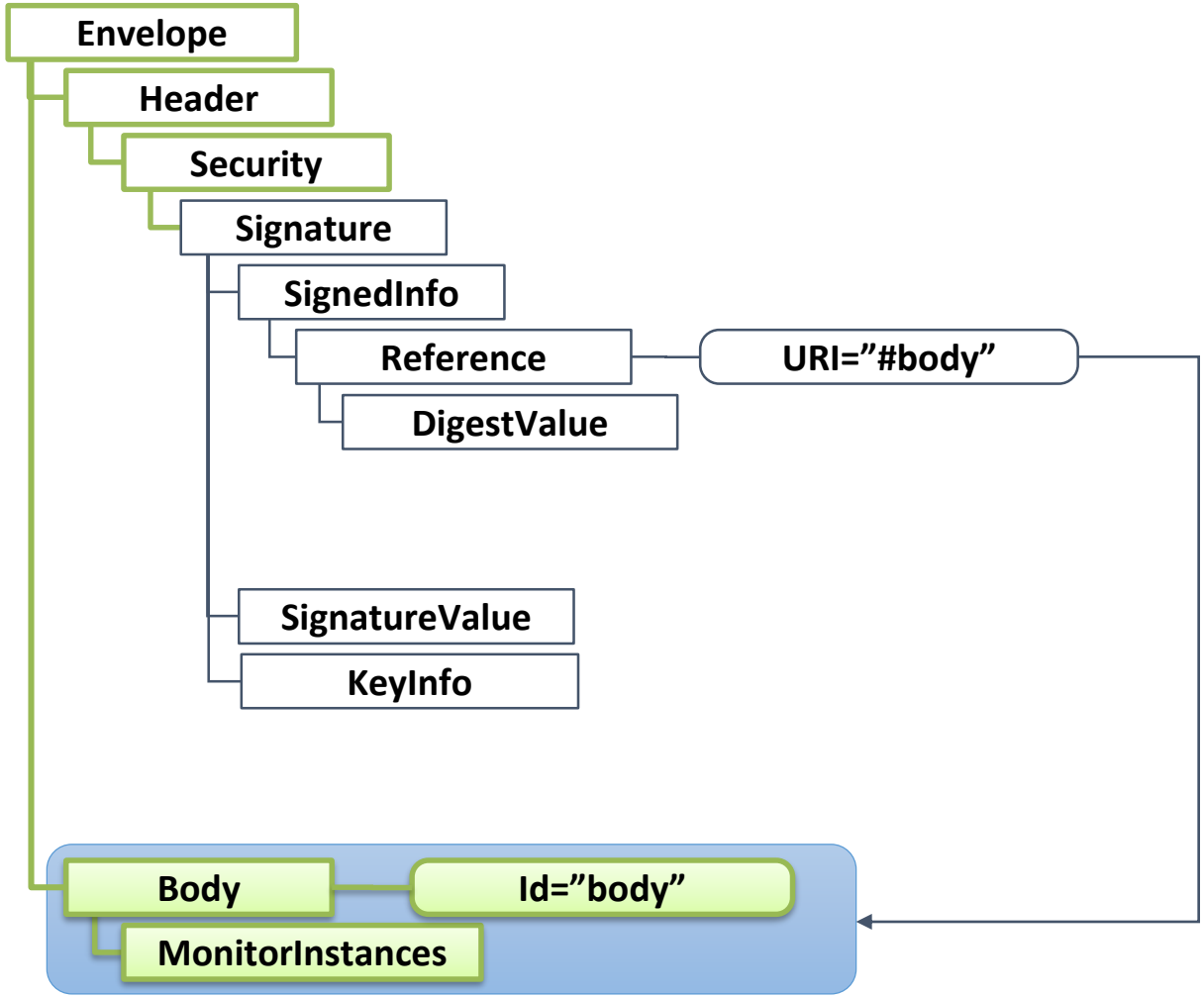
- Process only signed elements!



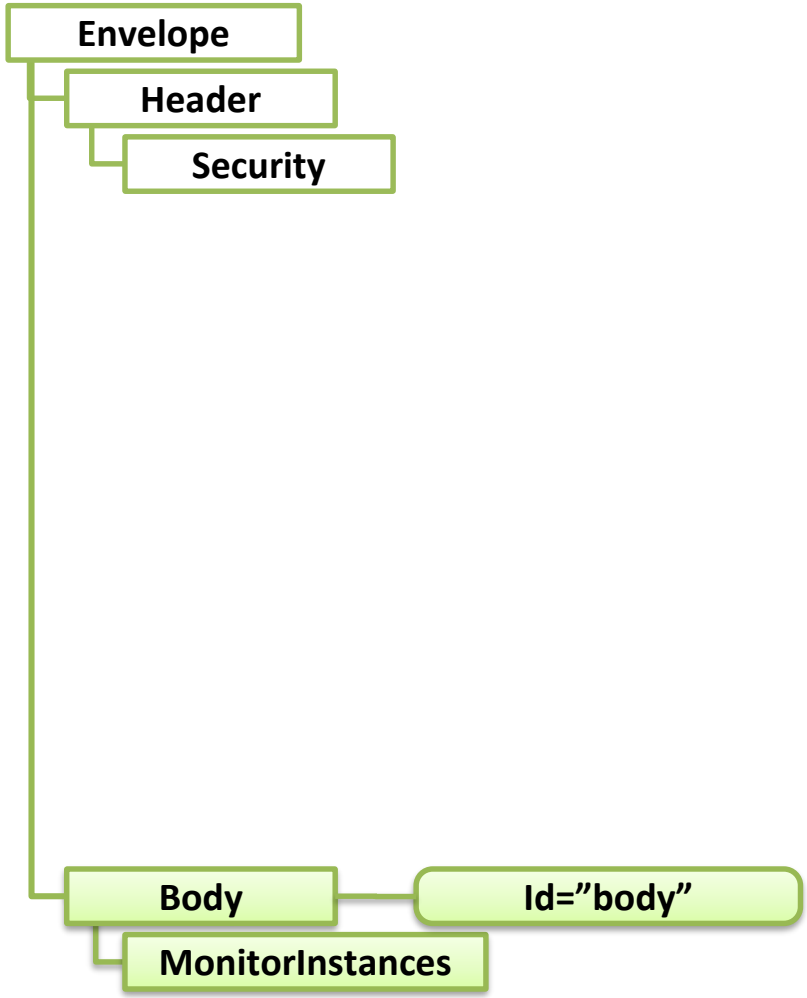
Overview

1. Cryptography Basics
2. XML Signature
3. HMAC Truncation Attack
4. XML Signature Wrapping
5. XML Signature Exclusion
6. SAML-based Single Sign-On

Signature Exclusion Attack



Signature Exclusion Attack



Worked against Amazon EC2 as well

https://www.comedywildlifephoto.com/

Overview

1. Cryptography Basics
2. XML Signature
3. HMAC Truncation Attack
4. XML Signature Wrapping
5. XML Signature Exclusion
6. SAML-based Single Sign-On

Motivation

- Many websites
- Many passwords
- Many identities

- How to solve this problem?

Motivation

- Solution: Single Sign-On

- Log in once at an Identity Provider
- Identity Provider proves to Service Providers that you have valid credentials

- Possible with SAML (Security Assertion Markup Language)

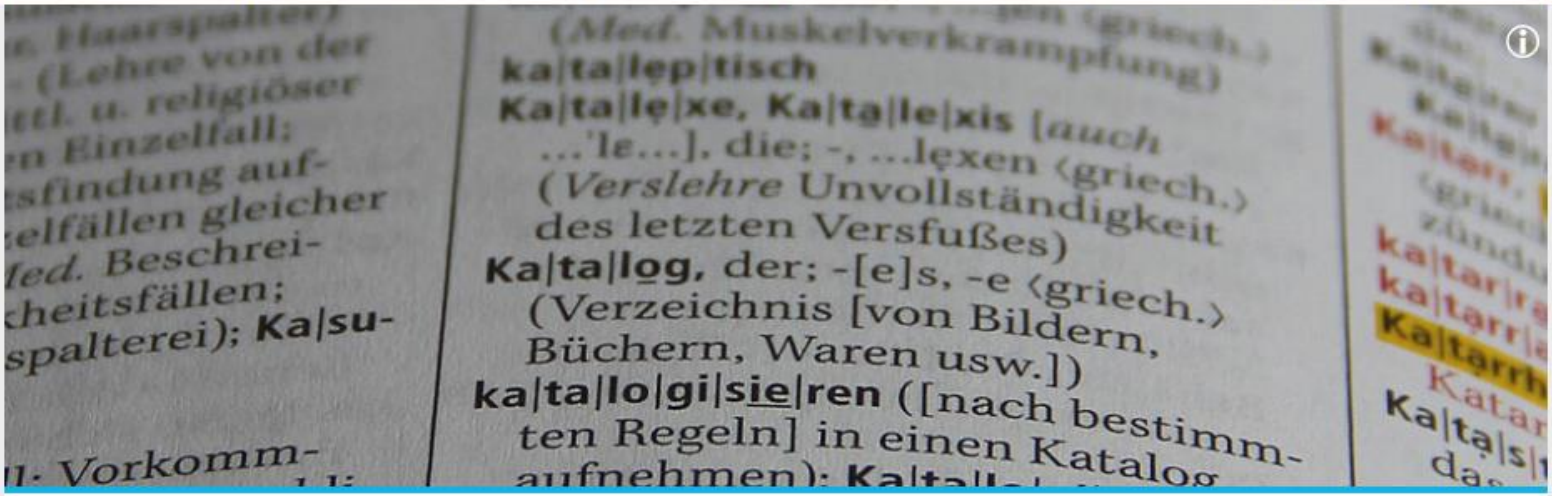
SCHNELLZUGRIFF ↓ KONTAKT DE | EN Suchen (Google)

UNIVERSITÄT PADERBORN

Identity Provider

UNIVERSITÄTSBIBLIOTHEK

RECHERCHE | NUTZEN UND LEIHEN | LERNORT | UNTERSTÜTZUNG | ÜBER UNS | A - Z



Universität Paderborn → Universitätsbibliothek Paderborn → Recherche → Hinweise zur Nutzung der elektronischen Angebote → Liste Service Provider Shibboleth

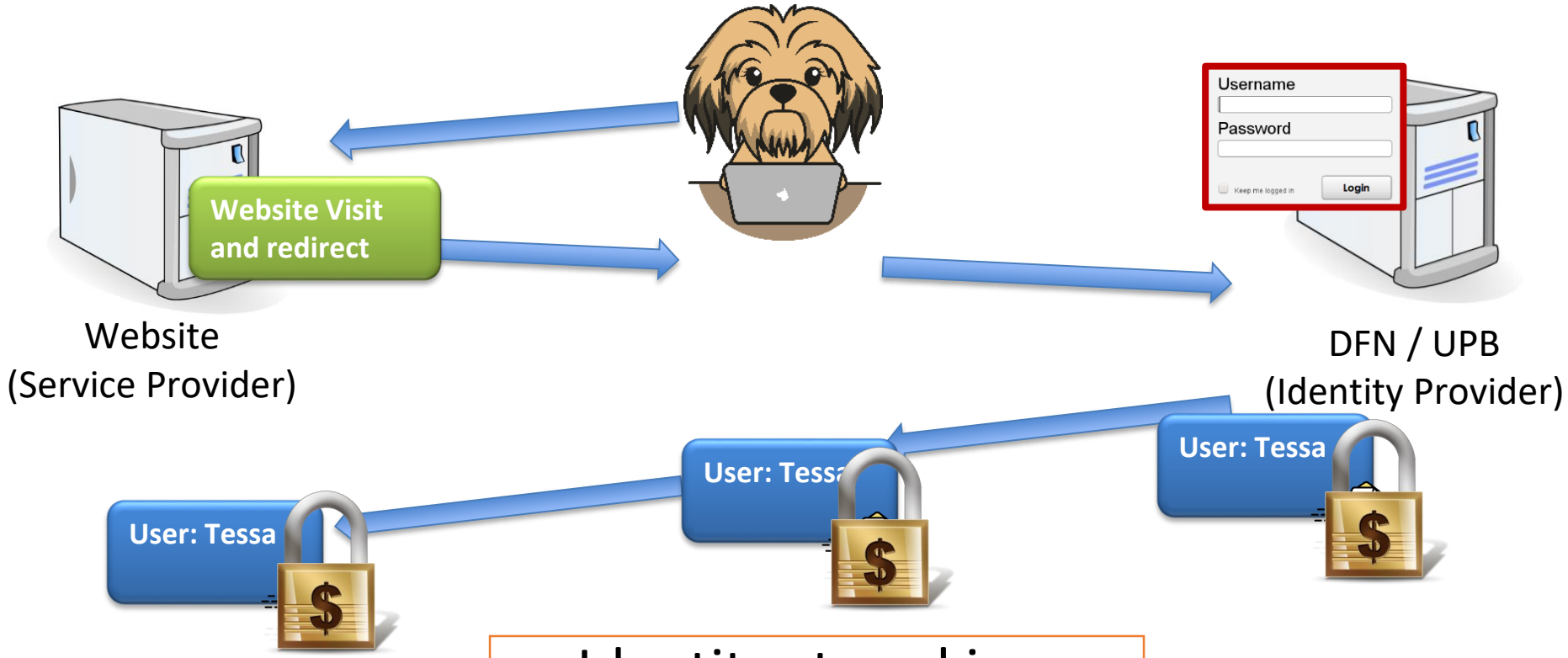
Service Provider (Verlage) mit Shibboleth (DFN-AAI) Profil für die Universität Paderborn

Service Provider / Verlage, die in der Liste einem Sternchen (*) gekennzeichnet sind, ermöglichen bei Zugriff aus dem SFX-Menü ein vereinfachtes Shibboleth-Anmeldeverfahren: nach dem Klick auf den SFX-Go-Button erfolgt die unmittelbare Weiterleitung auf die Uni-Account-Eingabemaske (eine vorherige Auswahl der Universität Paderborn als Institution ist nicht notwendig).

- American Association for the Advancement of Science - AAAS
- American Chemical Society
- American Institute of Physics
- American Society of Mechanical Engineers - ASME
- Annual Reviews
- Association for Computing Machinery - ACM
- Bloomsbury

Service Providers

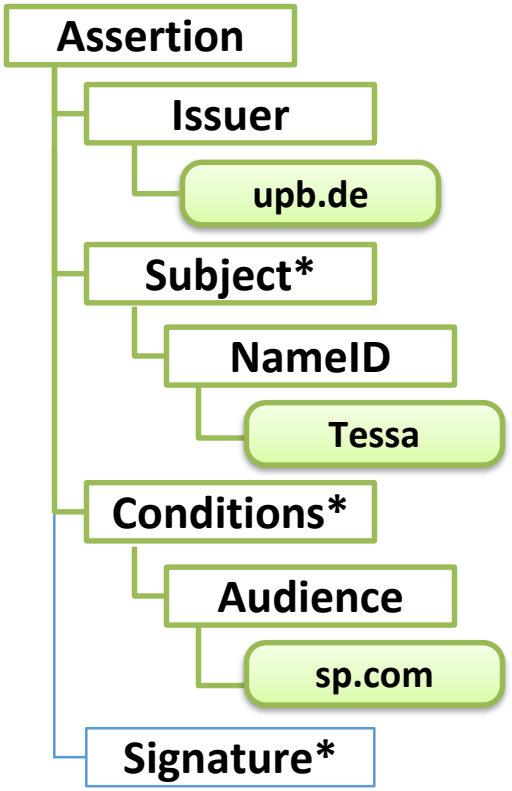
SAML-based Single Sign-On



Identity stored in a
SAML token / assertion

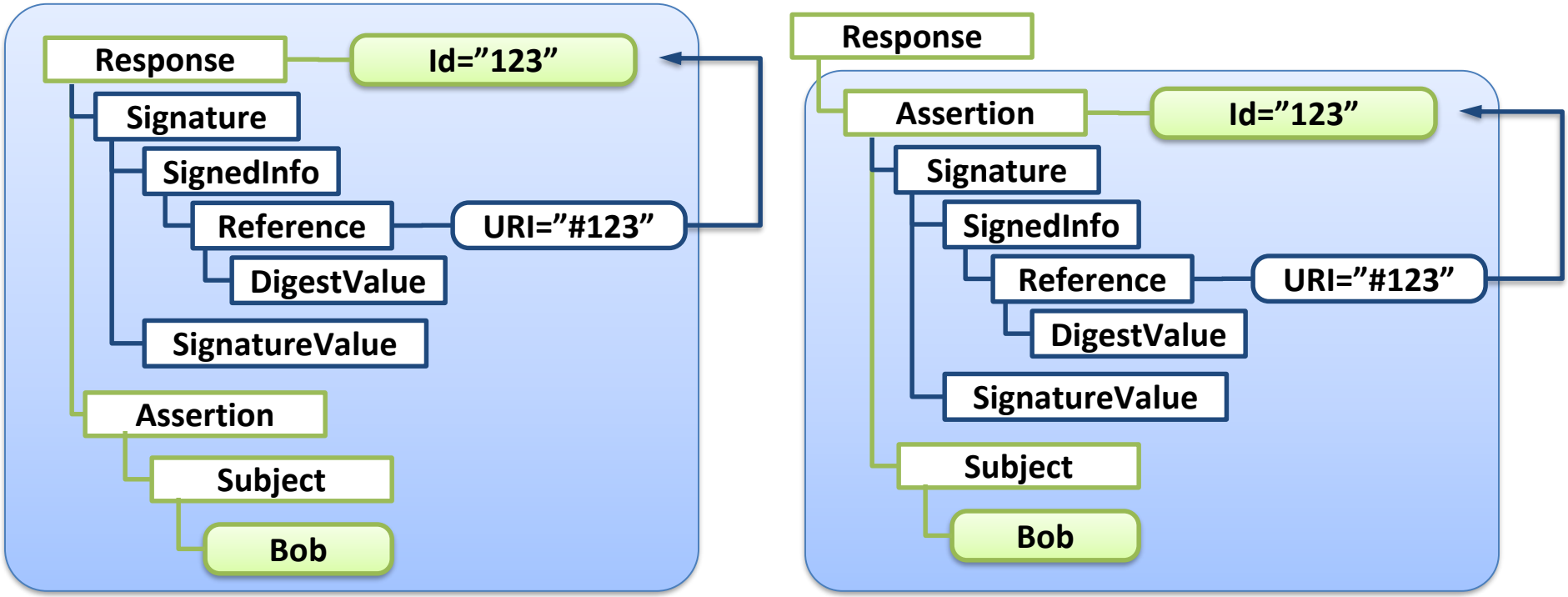
SAML Assertion

- XML Signatures used to sign SAML messages
- Breaking XML Signature == logging in as an arbitrary user



Securing SAML with XML Signature

- Two typical usages

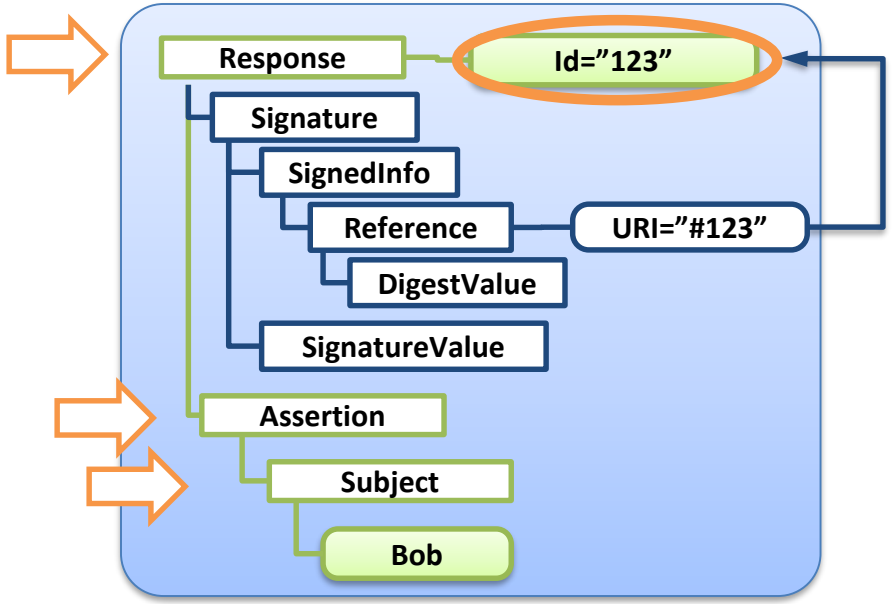


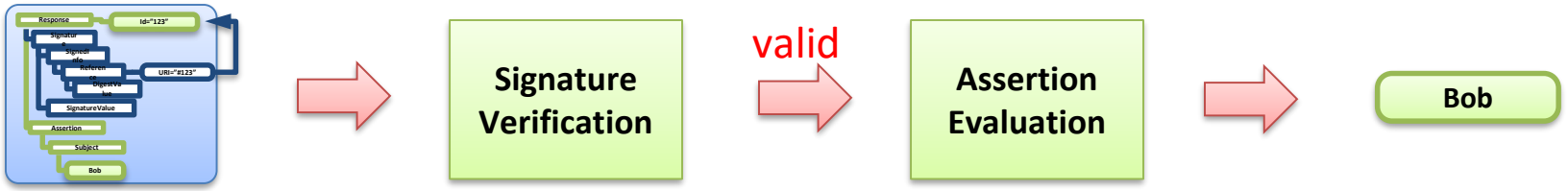
Securing SAML with XML Signature

- Naive (typical) processing:

Verifier Processor

1. Signature validation: **Id-based**
2. Assertion evaluation: **/Response/Assertion/Subject**

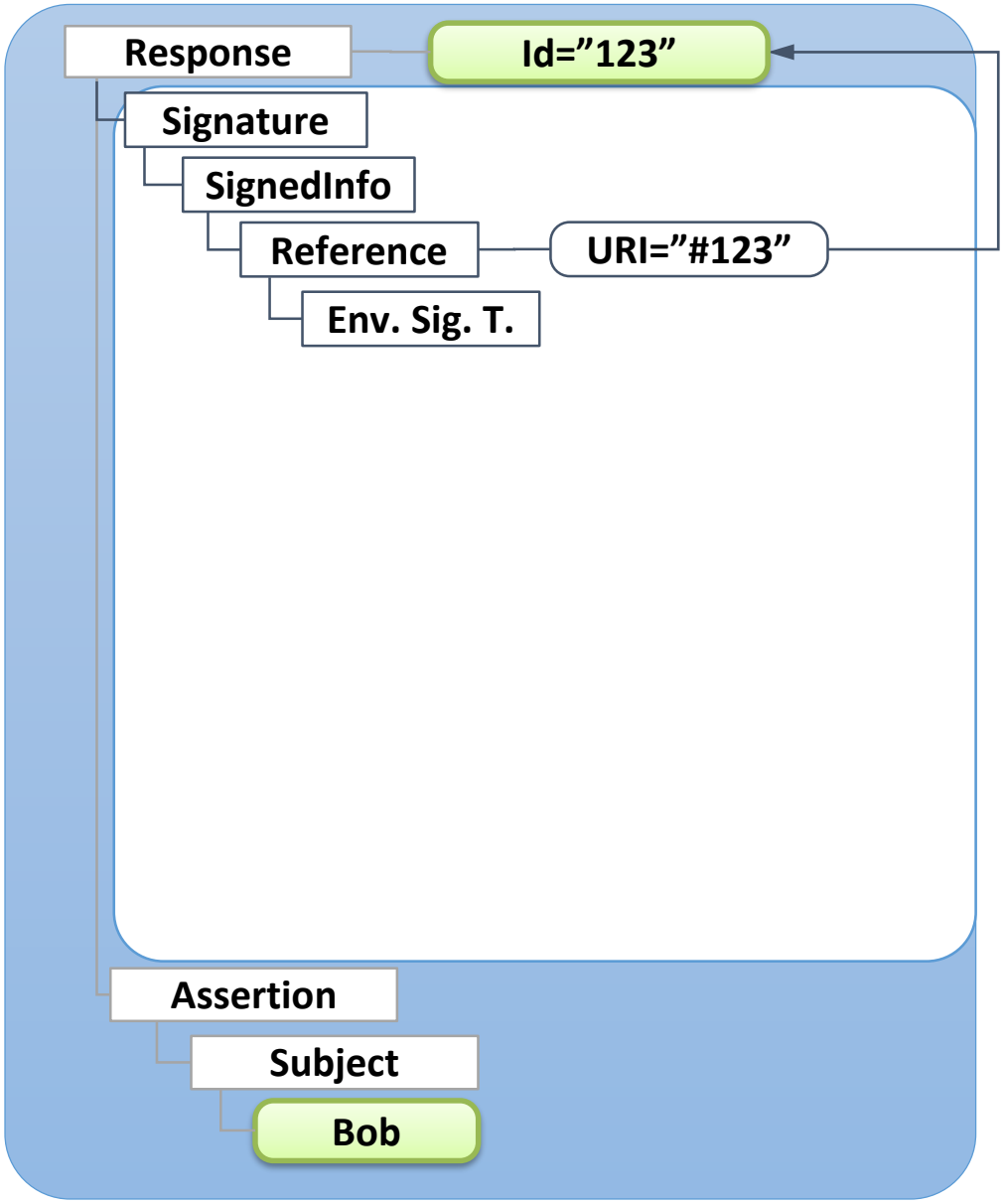




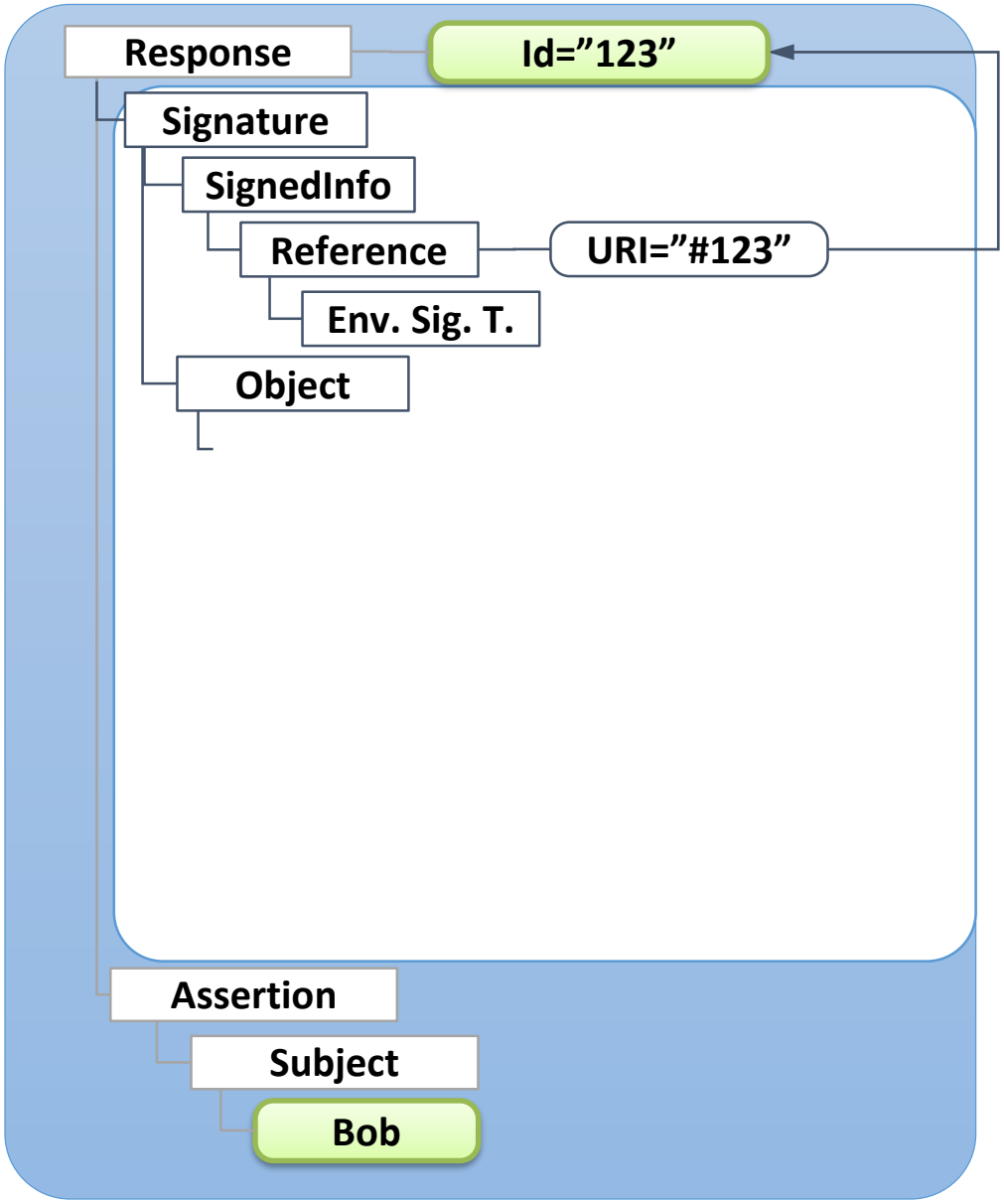
Can we use XML Signature Wrapping if the whole document is signed?

https://www.comedywildlifephoto.com/

XML Signature Wrapping – SAML

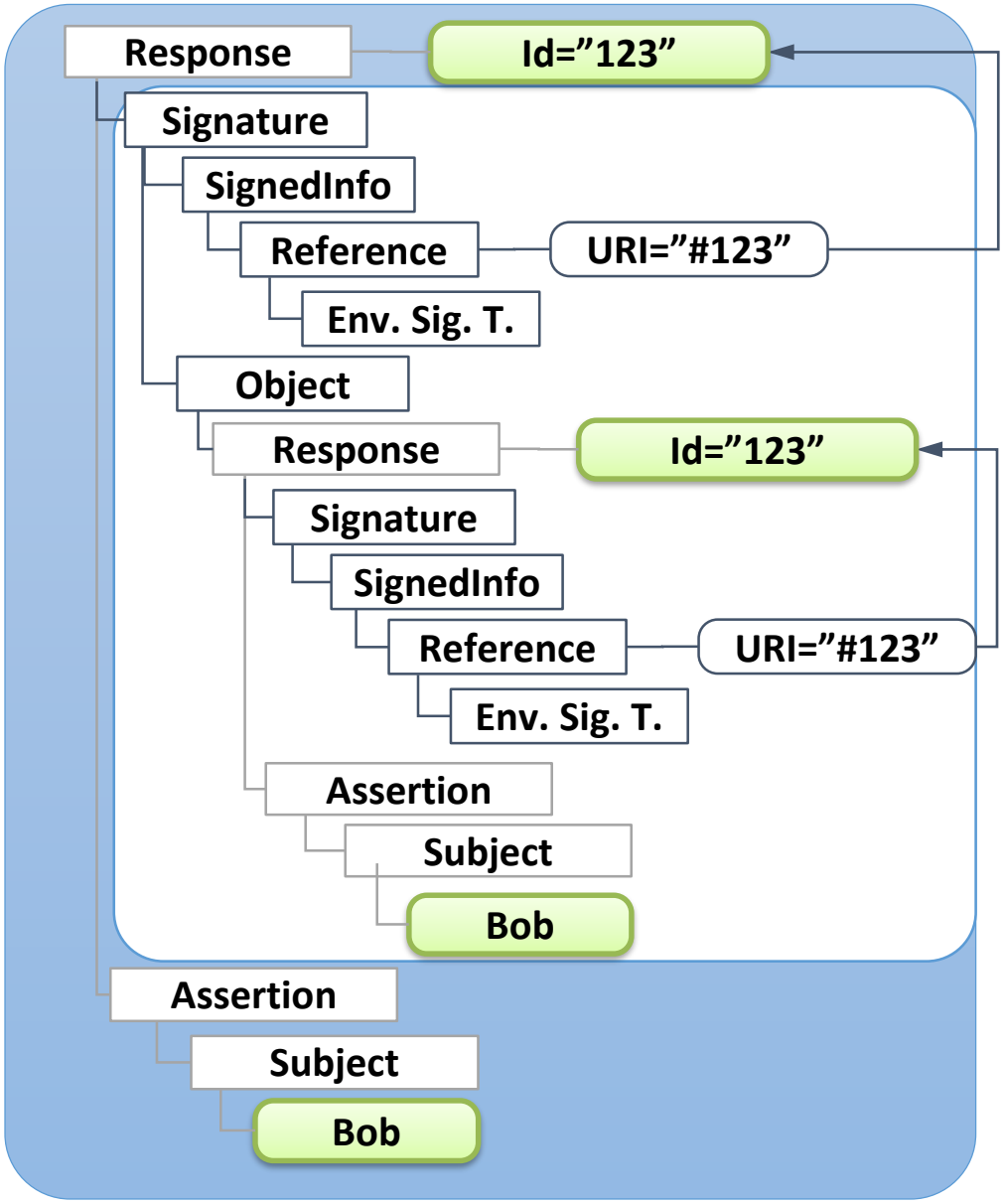


XML Signature Wrapping – SAML



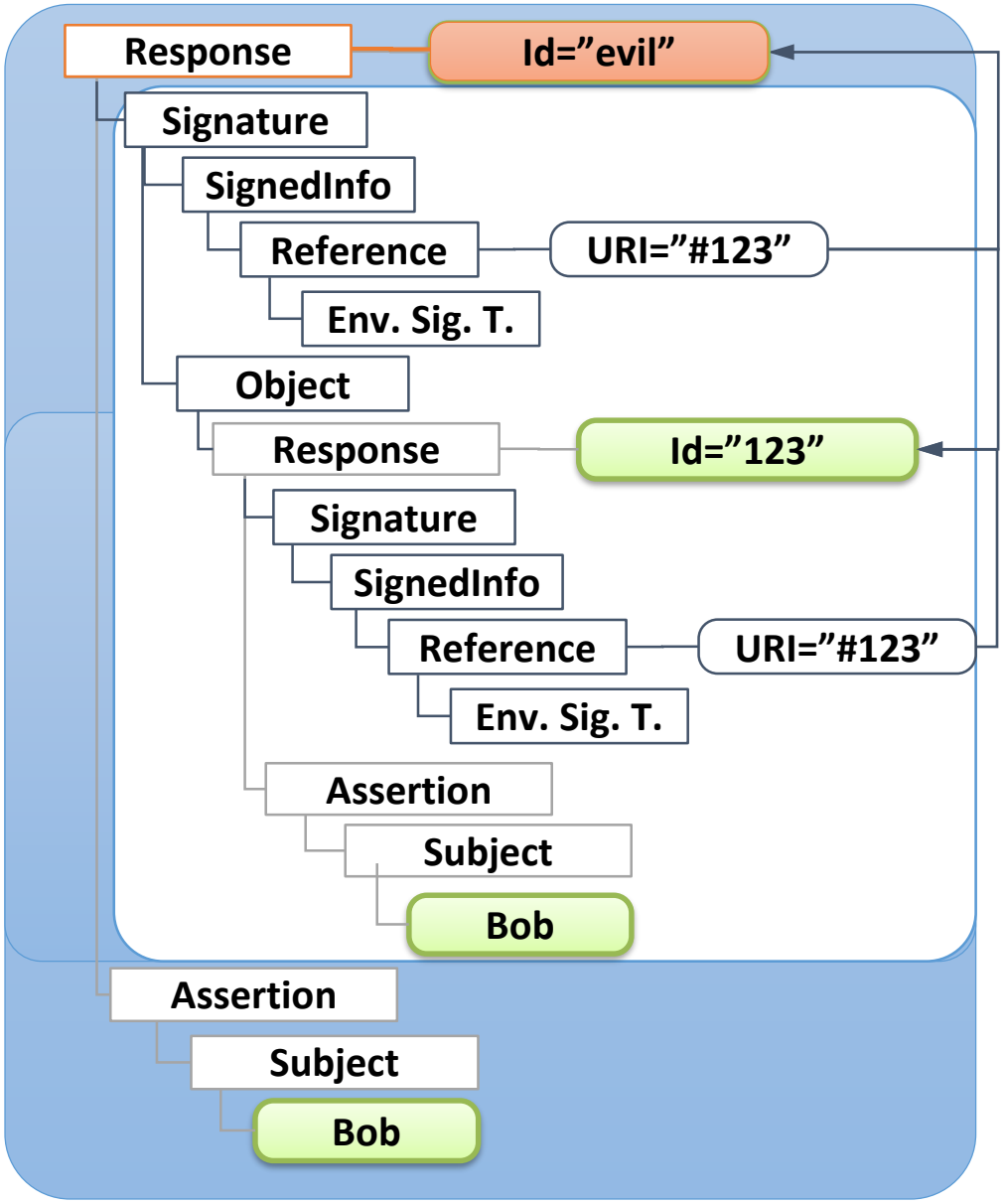
1. Find wrapper element

XML Signature Wrapping – SAML



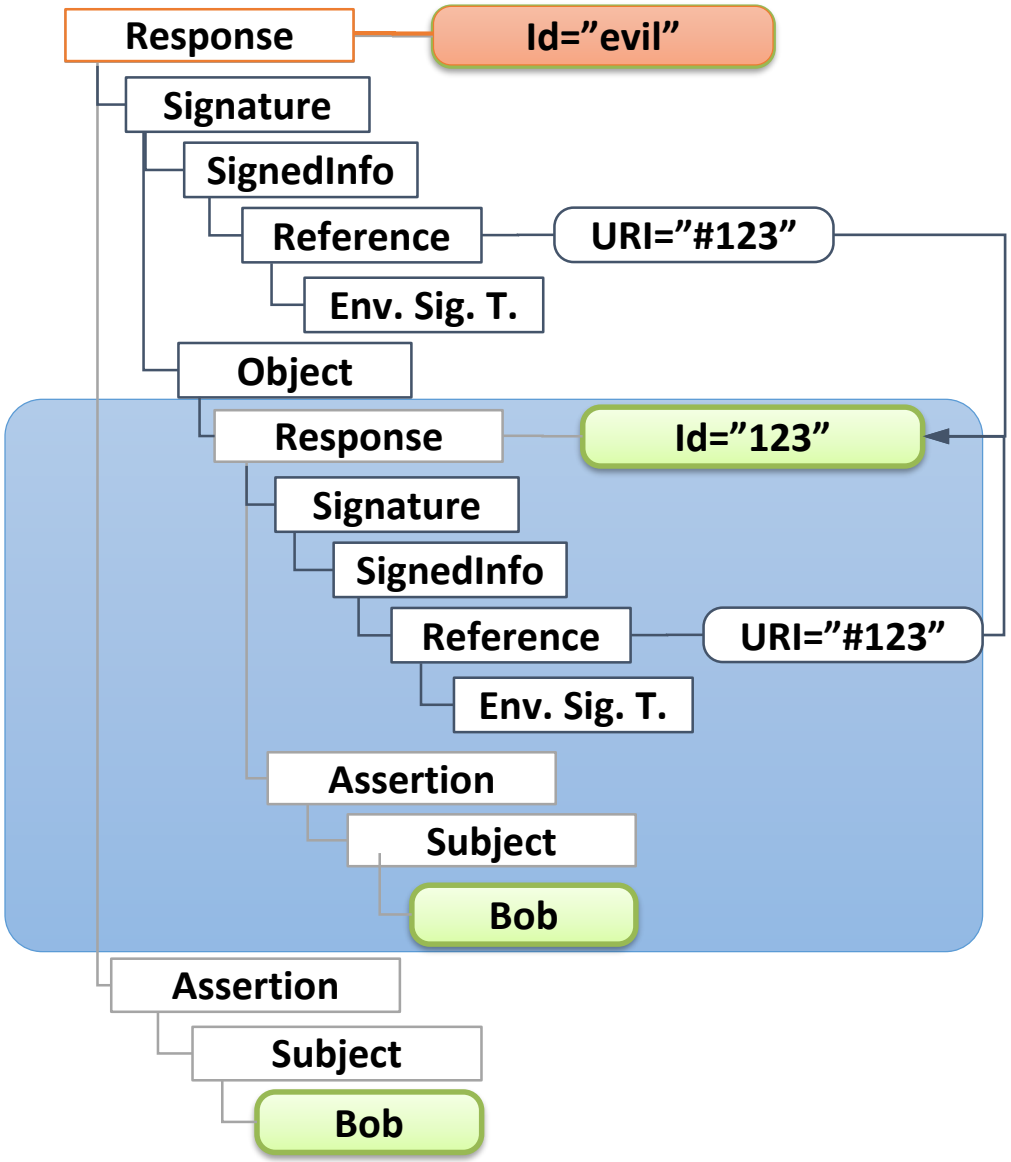
1. Find wrapper element
2. Copy original signed content into wrapper

XML Signature Wrapping – SAML



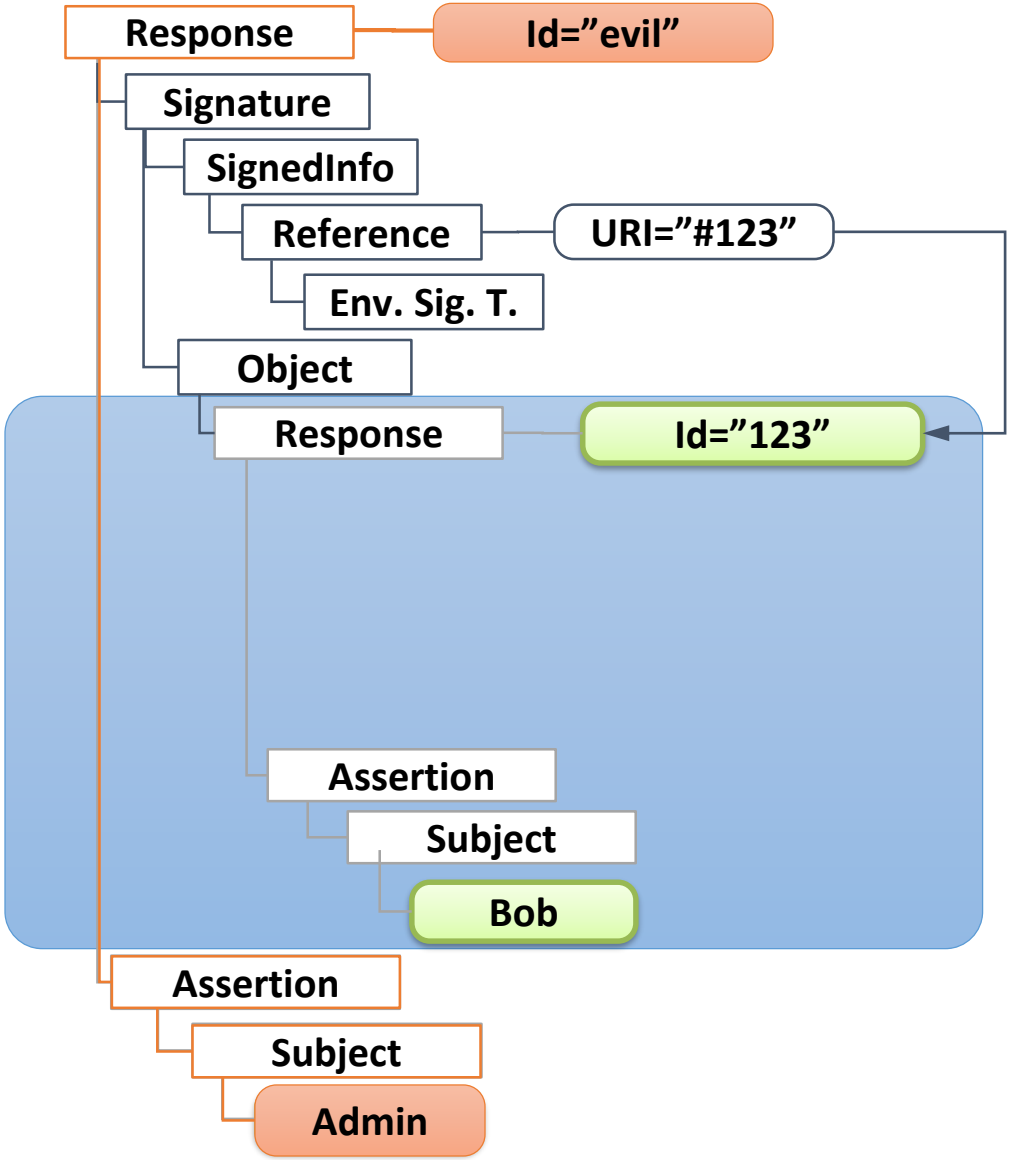
1. Find wrapper element
2. Copy original signed content into wrapper
3. Change the Id of the original element

XML Signature Wrapping – SAML



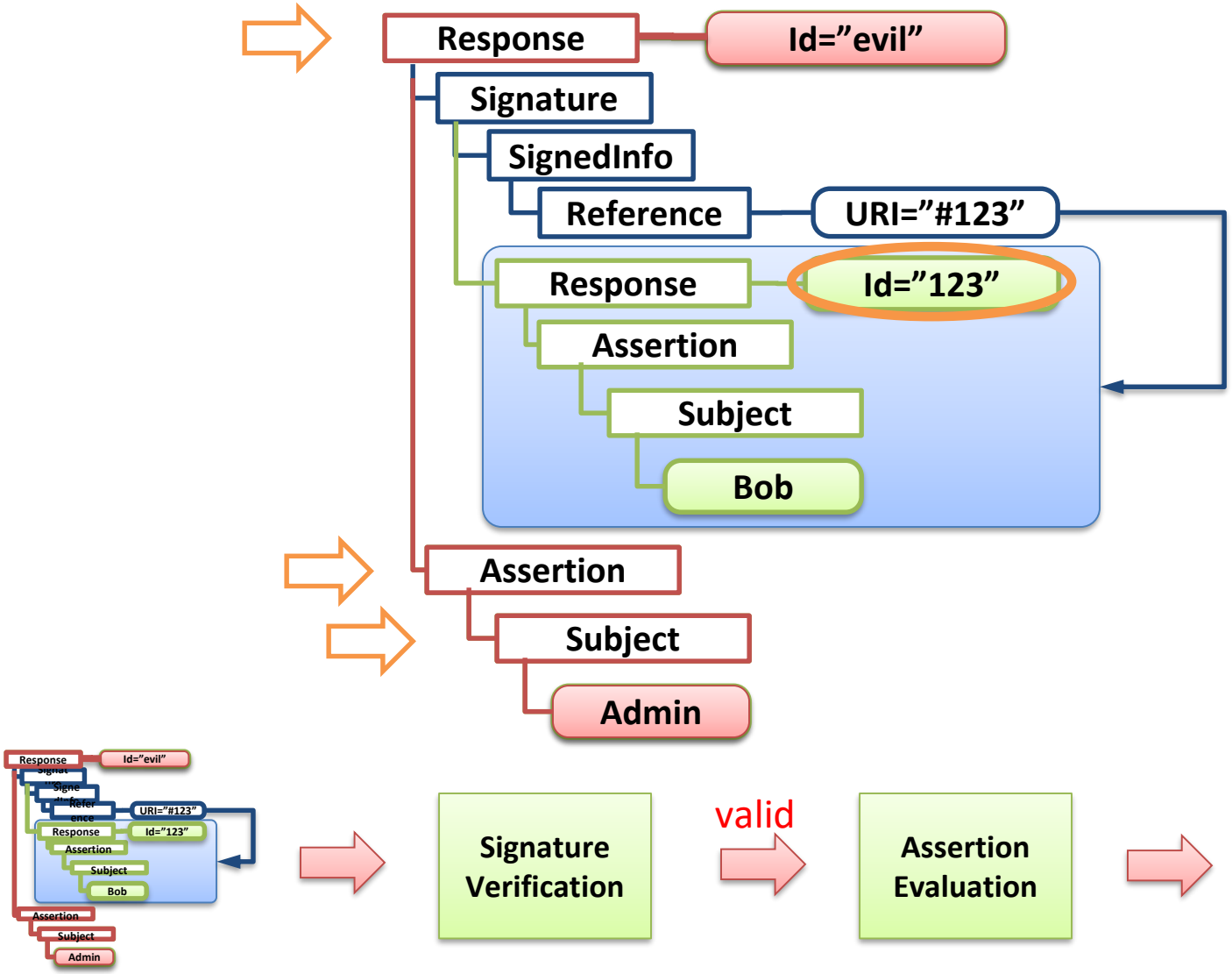
1. Find wrapper element
2. Copy original signed content into wrapper
3. Change the Id of the original element
4. Apply Enveloped Signature Transformation manually

XML Signature Wrapping – SAML

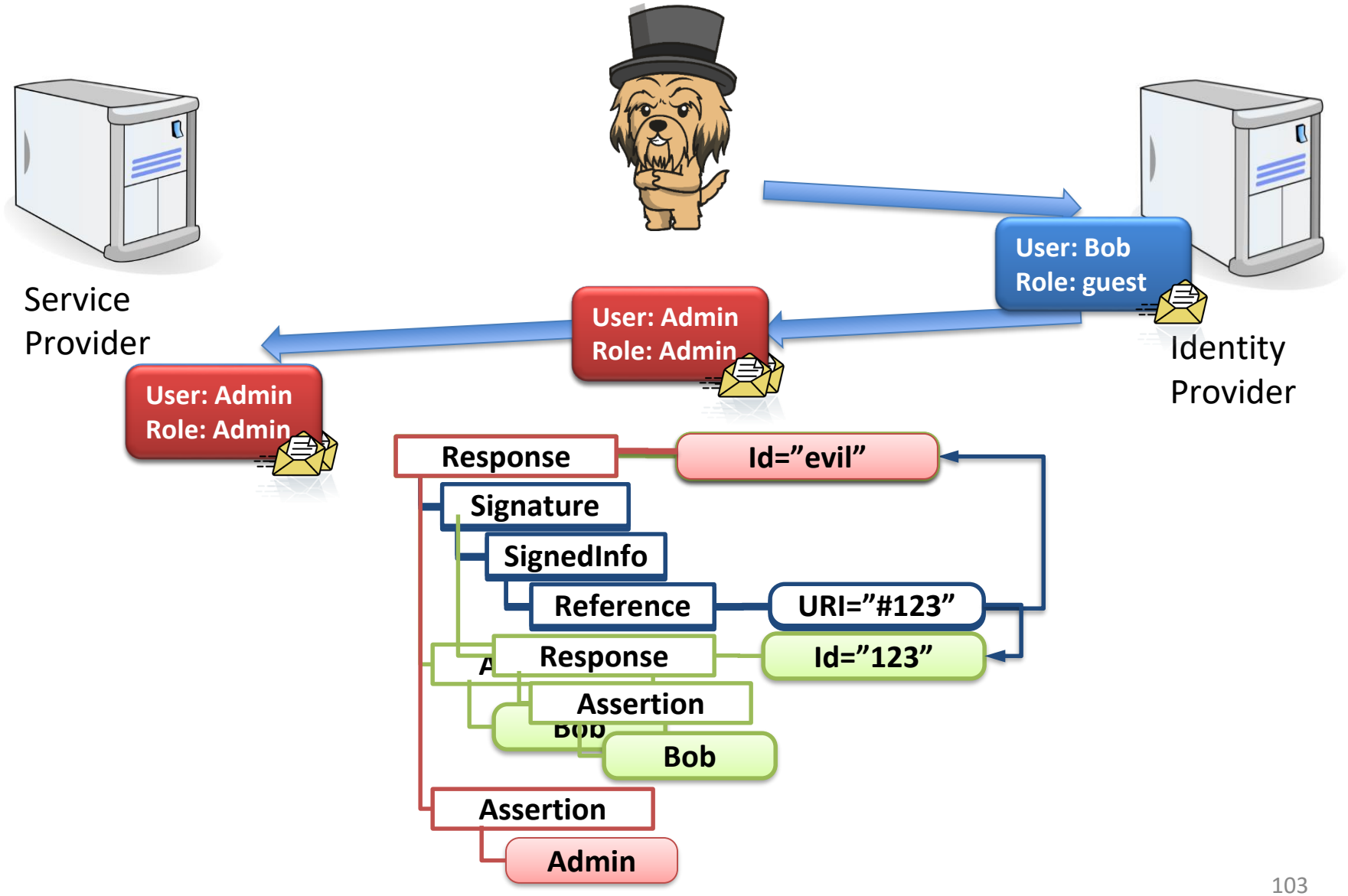


1. Find wrapper element
2. Copy original signed content into wrapper
3. Change the Id of the original element
4. Apply Enveloped Signature Transformation manually
5. Insert a new Assertion

XML Signature Wrapping



XML Signature Wrapping



Evaluation Results

Service Provider	AM1				AM2			AM3	
	∅Sig	CF	XXEA	XSLTA	RA	XSW	TRC	CInj	Summary
Salesforce	×	×	×	×	×	×	×	×	×
Google Apps	×	×	×	×	×	×	×	×	×
Zoho	×	×	×	×	×	✓	×	×	✓
Zendesk	×	×	×	×	×	×	✓	×	✓
Clarizen	✓	×	✓	×	✓	✓	✓	×	✓
SAManage	×	×	✓	×	×	✓	✓	✓	✓
Shiftplanning	×	×	✓	×	×	×	✓	✓	✓
Panorama9	×	×	×	×	×	×	✓	×	✓
UserVoice (Marketing)	×	×	×	×	×	×	✓	×	✓
Instructure	×	×	×	✓	✓	✓	✓	×	✓
The Resumator	×	×	✓	×	×	×	✓	×	✓
BambooHR	×	×	×	×	×	×	✓	✓	✓
AppDynamics	×	×	✓	×	✓	✓	✓	×	✓
IdeaScale	×	×	✓	×	×	×	×	✓	✓
Panopto	×	×	×	×	×	✓	✓	×	✓
TimeOffManager	×	×	✓	×	✓	✓	✓	×	✓
HappyFox	×	×	×	×	×	✓	✓	×	✓
SpringCM	×	×	×	×	×	✓	×	×	✓
ScreenSteps Live	×	×	✓	×	×	✓	✓	×	✓
LiveHive	×	×	✓	×	✓	✓	✓	×	✓
Howlr	×	×	×	×	×	×	✓	✓	✓
CA Service Management	×	×	✓	×	✓	×	✓	✓	✓
Total	1	0	10	1	6	11	17	6	20/ 22

Mainka et al.: Your Software at my Service (CCSW 2014)

Conclusions

- We saw many attacks exploiting XML parsing and data processing (no crypto know-how necessary)
- Creating interfaces between security and business logic is not that easy
- Many extensions make the standard open to new attacks (see HMACOutputLength)
- Best practices:
  - Follow security considerations
  - Disable extensions you do not need
- Other attacks:
  - XSLT attacks
  - Text content splitting with comments
  - Attacks on XML Encryption

Sources

- Jörg Schwenk: Lecture on XML and Web Service Security
- HMAC Truncation Vulnearability (http://www.w3.org/blog/2009/07/hmac-truncation-in-xml-signatu/)
- Brad Hill: A Taxonomy of Attacks against XML Digital Signatures & Encryption (https://www.isecpartners.com/media/11982/isec_hill_attackingxmlsecurity_handout.pdf)
- Juraj Somorovsky: On the Insecurity of XML Security, PhD Thesis, 2013
- Juraj Somorovsky, Mario Heiderich, Meiko Jensen, Jörg Schwenk, Nils Gruschka, Luigi Lo Iacono: All Your Clouds are Belong to us - Security Analysis of Cloud Management Interfaces. CCSW 2011
- Meiko Jensen, Christopher Meyer, Juraj Somorovsky, Jörg Schwenk: On the Effectivness of XML Schema Validation for Countering XML Signature Wrapping Attacks

Acknowledgements

- This is collaborative work with Juraj Somorovsky, Tibor Jager, Andreas Mayer, Christian Mainka, Jörg Schwenk, Marcus Brinkmann, and Vladislav Mladenov
- Tessa pictures by https://www.fiverr.com/tnhs_project